

ozGUI / ozLib

Ornstein–Zernike Solver Toolkit

including the multicomponent and polydisperse extension

ozGUI/ozLib project

September 26, 2026

Contents

I	The one-component toolkit	2
1	Overview	3
2	The Ornstein–Zernike Equation and its Discretisation	3
2.1	The OZ equation	3
2.2	Discrete grid: real space	4
2.3	Discrete Hankel transform	4
2.4	Grand-canonical thermodynamic quantities	4
3	Interaction Potentials	4
3.1	Hard Sphere	5
3.2	Lennard-Jones	5
3.3	Yukawa (screened Coulomb), with optional hard core	5
3.4	Star polymer potential (Likos & Harreis) [?]	5
3.5	Depletion potential (Oversteegen & Lekkerkerker) [?]	6
3.6	DLVO (electrostatic double-layer + van der Waals)	7
3.7	DLVO + hydration repulsion	7
3.8	Fermi distribution model (Likos)	7
3.9	Generalized Gaussian core model (GGCM- n)	7
3.10	Hard sphere + 3 additive Yukawa tails	7
3.11	Ionic microgel (Likos et al.)	7
3.12	Piecewise-constant hard-sphere shoulder/well (3 segments)	8
3.13	Penetrable sphere model (Likos, Watzlawek, Löwen) [?]	8
3.14	Soft sphere (inverse power law)	8
3.15	Parabolic (Hertzian) sphere	8
3.16	Square well	8
3.17	Sticky hard sphere (Baxter’s adhesive limit)	8
4	Closure Relations	9
4.1	Percus–Yevick (PY)	9
4.2	Hypernetted Chain (HNC)	9
4.3	Reference HNC (RHNC)	9
4.4	Modified HNC (MHNC)	9

4.5	Mean Spherical Approximation (MSA)	9
4.6	Rescaled MSA (RMSA)	10
4.7	Modified MSA (mMSA)	10
4.8	“Symmetric” MSA (SMSA)	10
4.9	Rogers–Young (RY) [?]]	10
4.10	HMSA (Zerah–Hansen) [?]]	10
4.11	Verlet [?] ?]	10
4.12	Martynov–Sarkisov (MS) [?]]	11
4.13	Vompe–Martynov (VM) [?]]	11
4.14	BPGG (Ballone–Pastore–Galli–Gazzillo) [?]]	11
4.15	CJVM (Charpentier–Jakse extension of VM) [?] ?]	11
4.16	BB (Bomont–Bretonnet) [?]]	11
4.17	Kovalenko–Hirata (KH) [?]]	11
4.18	Duh–Haymet (DH) [?]]	12
4.19	Choudhury–Ghosh (CG) [?]]	12
4.20	ZSEP (Lee) [?] ?] — hard spheres only	12
4.21	Euler–Rahman (EuRah) — not offered	13
4.22	Practical guidance: which closure for which potential	14
4.23	Literature validation against Monte Carlo: known accurate and inaccurate combinations	17
5	Analytical and Semi-Analytical Cross-Validation Tools	19
5.1	Percus–Yevick hard sphere (Wertheim–Thiele)	19
5.2	Baxter sticky hard sphere (idealised and finite-well)	19
5.3	Sharma–Sharma square well	19
5.4	Rescaled MSA (Hayter–Penfold)	19
5.5	One-Yukawa MSA (Blum–Høye)	20
5.6	ORPA (Pini–Parola–Reatto) for square wells	20
5.7	RFA (Rational Function Approximation)	20
6	Numerical Solver Algorithms	21
6.1	Picard iteration	21
6.2	Anderson acceleration (hand-written)	21
6.3	scipy Anderson / scipy Newton–Krylov	21
6.4	Biggs–Andrews	21
6.5	SUNDIALS KINSOL: Newton–Krylov (GMRES/FGMRES/TFQMR)	21
6.6	SUNDIALS KINSOL: Fixed-Point (Anderson), KIN_FP	22
6.7	Empirical solver comparison	22
7	The ozLib Python Library	23
7.1	Basic usage	23
7.2	Function signature	23
7.3	Example: potential with arguments and a parametrised closure	24
7.4	Example: long-range potential, extended grid	24
8	The ozGUI Application	24
8.1	Relationship to SASfit’s own Tcl GUI, and naming-convention differences	24
9	Worked Examples: Numerical vs. Analytical Validation	25
9.1	Hard-sphere Percus–Yevick: full validation across φ	26
9.2	Closure comparison at high density	26
9.3	Charged systems: RMSA and the MSA contact-value problem	27

9.4	Behaviour near difficult/critical parameter regimes	29
9.5	Speed	30
II	Multicomponent and polydisperse extension	33
10	Scope and design constraint	33
11	Multicomponent Ornstein–Zernike equations	33
11.1	Matrix form	33
11.2	Implementation	34
11.3	Public interface: 1-D curves preserved	34
12	Structure-factor conventions	34
13	Discretising a continuous size distribution	35
13.1	Moment matching	35
13.2	Verification	35
13.3	How many classes are needed?	35
14	Polydisperse hard-core Yukawa	36
14.1	Charge-parameterised form	36
14.2	Size dependence of the charge	36
14.3	MSA-consistent (z, K) form	37
15	Rogers–Young for mixtures	37
15.1	Closure	37
15.2	Thermodynamic consistency fixes α	37
15.3	Validation against the literature	38
15.4	Cross-validation against the analytic MSA path	39
16	Solver strategy	39
17	The SASfit plugin	40
18	New GUI tabs	42
19	Generic polydisperse models: any potential, any closure	44
19.1	Reusing eighteen setters unmodified	44
19.2	Quadrature rule per distribution	45
19.3	Structure factor and form factor need different resolutions	45
19.4	Length scale and the radius convention	46
20	Closure suite extended and cross-validated	46
21	Consistency search for a general potential	47
22	Defects found during validation	48
23	Regression testing	50
24	Solver reliability: multiple solutions and verification	51
24.1	The Ornstein–Zernike equation has several solutions	51
24.2	What the library does about it	51

24.3 Solver behaviour, measured	52
24.4 A converged flag is a claim; the residual is evidence	52
25 Fitting measured data	53
25.1 Where the hard core sits on the grid	58
26 Known limitations	59

Part I

The one-component toolkit

Abstract

This document is the reference manual for `ozGUI` and `ozLib`, a Python re-implementation of SASfit's Ornstein–Zernike (OZ) equation solver (`src/sasfit_oz/` in the SASfit source tree). It documents, with exact mathematical formulas, every interaction potential, every closure relation, and every numerical solver algorithm currently implemented, together with the underlying Hankel-transform-based discretisation scheme and the two ways of using the toolkit: as an interactive Tkinter GUI (`oZgui.py`) and as a plain importable Python library (`ozLib.py`). Formulas throughout were ported and verified directly against SASfit's own C source (`src/sasfit_oz/sasfit_oz_solver.c`, `src/plugins/*`), and, where an independent literature source was available, cross-checked against it as well; deviations, known limitations, and one deliberately-excluded closure are documented explicitly rather than silently omitted. `oZgui.py` is specifically a from-scratch Tkinter port of SASfit's own built-in Tcl/Tk Ornstein–Zernike solver window; Section 8.1 cross-references the two GUIs' naming conventions directly against both source files, rather than assuming a 1:1 correspondence.

1 Overview

ozGUI/ozLib solves the Ornstein–Zernike (OZ) integral equation for a single-component, isotropic fluid of spherically symmetric particles, given

- a pair interaction potential $U(r)$ (Section 3),
- a closure relation connecting the direct correlation function $c(r)$ to the total correlation function $h(r) = g(r) - 1$ (Section 20),
- a volume fraction φ , and
- a numerical solver algorithm that finds the self-consistent fixed point of the resulting nonlinear system (Section 16).

The output is the pair correlation function $g(r)$, the direct correlation function $c(r)$, the static structure factor $S(q)$, and several auxiliary quantities (the cavity function $y(r)$, the bridge function $B(r)$, the indirect correlation function $\Gamma(r) = h(r) - c(r)$, and the reduced potential $U(r)/k_B T$).

Two front ends share the exact same computational core (`ozLib.py`), so results obtained through either are identical by construction:

- `ozgui.py` — an interactive Tkinter application with potential/closure/solver dropdowns, a 9-tab plot panel, run history, and Save/Load/ASCII/CSV/Excel/PNG export (Section 8).
- `ozLib.py` — a plain Python function, `ozLib.solve(...)`, usable directly in scripts without any GUI dependency (Section 7).

2 The Ornstein–Zernike Equation and its Discretisation

2.1 The OZ equation

For an isotropic, single-component fluid the Ornstein–Zernike equation reads, in real space,

$$h(r) = c(r) + \rho \int c(|\mathbf{r} - \mathbf{r}'|) h(r') d\mathbf{r}', \quad (1)$$

where ρ is the number density and the convolution is over all space. Introducing the indirect correlation function $\Gamma(r) \equiv h(r) - c(r)$, and Fourier–Bessel (Hankel) transforming (exploiting isotropy to reduce the 3D Fourier transform to a 1D sine transform), the OZ equation becomes purely algebraic in reciprocal space,

$$\hat{\Gamma}(q) = \frac{\hat{c}(q)}{1 - \rho \hat{c}(q)} - \hat{c}(q), \quad (2)$$

where $\hat{f}(q)$ denotes the (3D, spherically-symmetric) Fourier transform of $f(r)$. The OZ equation alone relates two unknowns, $c(r)$ and $h(r)$ (or equivalently $\Gamma(r)$); a second, independent relation — the *closure* — is required to close the system. Every closure implemented here has the general form

$$c(r) = f(\Gamma(r), U(r)) \quad (3)$$

for some function f specific to the closure (Section 20). The full problem is therefore a fixed-point problem in $\Gamma(r)$ (or, in an equivalent formulation used by some solvers here, in $c(r)$ and $h(r)$ jointly): starting from a trial $\Gamma(r)$, the closure gives $c(r)$, the OZ equation (Eq. 2) gives a new $\Gamma(r)$, and a solver algorithm (Section 16) drives this map to self-consistency.

2.2 Discrete grid: real space

The real-space grid is a plain equal-spaced grid of $N = \text{numberOfRadialSamplingPoints}$ points,

$$r_i = i \Delta r, \quad i = 1, 2, \dots, N, \quad (4)$$

starting at Δr rather than 0 (division by r appears in the Hankel transform below, so $r = 0$ is avoided by construction rather than special-cased). The hard-core diameter σ is placed at grid index `hardSphereDiameterInPoints` (called n_σ below), i.e.

$$\sigma = n_\sigma \Delta r, \quad \text{total range } r_{\max} = (N - 1) \Delta r = \frac{N - 1}{n_\sigma} \sigma. \quad (5)$$

Both N and n_σ are configurable (constructor keyword arguments `numberOfRadialSamplingPoints`, `hardSphereDiameterInPoints` on every solver class, and identically-named arguments on `ozLib.solve()`); the historical defaults are $N = 4096$, $n_\sigma = 100$, giving $r_{\max} \approx 41 \sigma$ and $\Delta r = 0.01 \sigma$. Increasing N at fixed n_σ extends the range without losing near-contact resolution — important for slowly-decaying, long-range potentials (e.g. small- κ Yukawa/DLVO tails), where a too-short grid truncates the potential before it has actually decayed to zero, distorting $S(q)$ at low q (this was verified directly: a $4\times$ extension of the default grid reduced a Yukawa potential's residual value at the grid edge from $\mathcal{O}(10^{-4}) k_B T$ to $\mathcal{O}(10^{-10}) k_B T$, and changed $S(q \rightarrow 0)$ by about 33%).

2.3 Discrete Hankel transform

For an isotropic function $f(r)$ the 3D Fourier transform reduces to

$$\hat{f}(q) = \frac{4\pi}{q} \int_0^\infty r f(r) \sin(qr) dr, \quad (6)$$

which is implemented here as a type-I discrete sine transform (DST) of $r_i f(r_i)$, following the same construction documented in SASfit's own theory notes (equations 5.18/5.19 there):

$$\hat{f}(q_k) = \frac{2\pi \Delta r^2}{\Delta q} \cdot \frac{\text{DST-I}[r_i f(r_i)]_k}{k}, \quad \Delta q = \frac{\pi}{(N + 1) \Delta r}, \quad q_k = k \Delta q. \quad (7)$$

The inverse transform is the same operation up to a normalisation factor ($\Delta q^3 / [(2\pi)^3 \Delta r^3]$), since the Hankel transform pair is (up to constants) self-inverse. Both directions are implemented once, in `0ZfixpointOperator.hankelTransform()/inverseHankelTransform()`, and shared by every closure and every solver.

2.4 Grand-canonical thermodynamic quantities

For the hard-sphere/Percus–Yevick case, the exact grand-canonical compressibility-route structure factor at $q = 0$ is available in closed form (used as an internal consistency check, e.g. in RMSA's own bisection, see Section 4.6):

$$S(q=0) = \frac{(1 - \varphi)^4}{(1 + 2\varphi)^2}. \quad (8)$$

3 Interaction Potentials

Every potential is specified through its Boltzmann factor $\exp[-U(r)/k_B T]$ (stored as `boltzmannOfP2Potential`), evaluated on the grid described above; all energies are in $k_B T$ units and

all lengths in units where the hard-core diameter is $\sigma = n_\sigma \Delta r$. Potentials that support the repulsive/attractive potential split needed by switching-function closures (Rogers–Young, HMSA, and the Gstar-based bridge closures of Section 20) additionally populate the arrays `repulsivePartOfP2Ppotential` and `attractivePartOfP2Ppotential`. All 19 potentials below are available via `setPotentialByName(name, *args)` or directly as `set<Name>Potential(...)` – in 18 subsections, not 19, since Section 3.4 covers both of the star polymer potential’s two distinct setters (`setStarPolymerHighFPotential()`, `setStarPolymerLowFPotential()`) together, one formula each. Two further setters, `setPolydispersePotential()` and `setPolydisperseHard-CoreYukawaPotential()`, are wrappers that build a MULTICOMPONENT system from one of the below and are described with the polydisperse machinery rather than here, bringing the solver’s total to 21.

Note on counting, and on a genuine fix this made: the GUI’s own potential dropdown (Table 2, Section 8.1) lists 18 named entries. An earlier version of this port had only 17 setters – `setStarPotential()` implemented only the star polymer potential’s $f \geq 10$ formula, silently applied regardless of functionality, with no $f \leq 10$ counterpart at all, unlike the original Tcl GUI’s two separate `StarPolymer (f>10)` / `StarPolymer (f<10)` dropdown entries. Checking `sasfit_oz_potential_star_Likos.c` directly (rather than assuming the single existing formula covered every f) found this is a genuinely different second formula (`U_Star2`, Section 3.4), not just the same one evaluated at a smaller argument – now added as `setStarPolymerLowFPotential()`, matching the Tcl GUI’s count exactly. `Depl-Sph-Sph/Depl-Sph-Discs/Depl-Sph-Rods` similarly appear as three separate GUI choices against one Depletion subsection here – checked the same way the star polymer case was: confirmed to be a similarly-missing split (see Section 3.5’s own **Known gap** note), not a single shared formula with different constant substitutions, though unlike star polymer this one was found but not fixed in this pass.

3.1 Hard Sphere

$$\exp[-U(r)/k_B T] = \begin{cases} 0 & r < \sigma \\ 1 & r \geq \sigma. \end{cases} \quad (9)$$

3.2 Lennard-Jones

$$U(r)/k_B T = 4\varepsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right], \quad (10)$$

with ε in $k_B T$ units. The potential is split at its minimum ($r_{\min} = 2^{1/6} \sigma$) into a repulsive part ($r < r_{\min}$) and an attractive part ($r \geq r_{\min}$), for use by switching-function closures.

3.3 Yukawa (screened Coulomb), with optional hard core

$$U(r)/k_B T = K \frac{\sigma}{r} e^{-r/\lambda}, \quad (11)$$

where λ is the screening length (in units of σ) and K the contact interaction strength; a hard core can optionally be added (`doAddHardSphere`), in which case the exponential and $1/r$ factors are shifted independently as in SASfit’s own construction, and the potential is set to ∞ (Boltzmann factor 0) for $r < \sigma$.

3.4 Star polymer potential (Likos & Harreis) [?]]

Two genuinely different formulas, one for each of two functionality regimes, matching `U_Star1` and `U_Star2` in `sasfit_oz_potential_star_Likos.c` respectively. This port calls them

`setStarPolymerHighFPotential()` and `setStarPolymerLowFPotential()`; an earlier version only had the first of these, silently applying it regardless of f .

High functionality, $f \geq 10$ (exponential decay outside the core):

$$U(r)/k_B T = \begin{cases} \frac{5}{18} f^{3/2} \left[-\ln \frac{r}{\sigma} + \frac{1}{1 + \sqrt{f}/2} \right] & r \leq \sigma \\ \frac{5}{18} f^{3/2} \frac{1}{1 + \sqrt{f}/2} \frac{\sigma}{r} \exp \left[-\frac{\sqrt{f}(r - \sigma)}{2\sigma} \right] & r > \sigma. \end{cases} \quad (12)$$

Low functionality, $f \leq 10$ (Gaussian decay outside the core, with its own $\tau(f)$ interpolation):

$$\tau(f) = \tau_2 + \frac{\tau_5 - \tau_2}{3} f, \quad \tau_2 = \frac{1.03}{\sigma}, \quad \tau_5 = \frac{1.12}{\sigma}, \quad (13)$$

$$U(r)/k_B T = \begin{cases} \frac{5}{18} f^{3/2} \left[-\ln \frac{r}{\sigma} + \frac{1}{2(\tau\sigma)^2} \right] & r \leq \sigma \\ \frac{5}{18} f^{3/2} \frac{1}{2(\tau\sigma)^2} \exp[-\tau^2(r^2 - \sigma^2)] & r > \sigma. \end{cases} \quad (14)$$

Here $f = \text{NumberOfArms}$ is the number of arms of the star polymer (the same argument name on both setters). The two branches do *not* meet smoothly at $f = 10$ – checked directly: $\sim 28\%$ relative difference in $U(r = \sigma^+)$, about $30\times$ apart by $r = 2\sigma$. This is a genuine property of the original paper’s own two-regime fit (each branch is only claimed accurate within its own stated range), not a bug in either formula or this port; $f = 10$ is where Likos & Harreis recommend switching approximations, not a point the two forms were designed to agree at exactly. A caller choosing between the two should therefore pick whichever branch matches their actual f , not assume either extrapolates sensibly across the boundary.

3.5 Depletion potential (Oversteegen & Lekkerkerker) [?]

Two-sphere overlap-volume depletion potential between big spheres (diameter $\sigma_1 = \sigma$, the species this OZ solve is for) induced by small depletant spheres (diameter $\sigma_2 = \text{sigmaRatio} \cdot \sigma_1$, number density ρ_2 set via the depletant volume fraction `phi2`):

$$U(r)/k_B T = -\rho_2 \frac{\pi \sigma_1^3 (1 + r_t)^3}{6} \left[1 - \frac{3r}{2(1 + r_t)\sigma_1} + \frac{r^3}{2(1 + r_t)^3 \sigma_1^3} \right], \quad \sigma_1 \leq r < \sigma_1 + \sigma_2, \quad (15)$$

with $r_t = \sigma_2/\sigma_1$, and $U = 0$ for $r \geq \sigma_1 + \sigma_2$; hard core for $r < \sigma_1$.

Known gap, found while cross-checking the star polymer split of Section 3.4 prompted a closer look at this file too: `sasfit_oz_potential_depletion.c` actually contains *four* distinct formulas, not one – the generic `U_Depletion` above, plus `U_DepletionOfSpheresBySpheres`, `U_DepletionOfSpheresByDiscs`, and `U_DepletionOfSpheresByRods`, matching the Tcl GUI’s own three named dropdown entries (`Depl-Sph-Sph/Depl-Sph-Discs/Depl-Sph-Rods`, Table 2, Section 8.1) – each geometrically distinct (sphere depleted by spheres/discs/rods respectively, the disc and rod cases involving genuinely different overlap-volume geometry, e.g. an arcsin term for discs) and parametrised differently from `U_Depletion` (`sigma_large`, `sigma_small/D/L`, and a number density `nb` directly, rather than `sigma1/sigma2/a` volume fraction `PHI2`). This Python port currently only has the generic `U_Depletion` form – unlike the star polymer fix of Section 3.4, this was found but deliberately *not* fixed in this pass (out of the scope of what was asked, and a larger, riskier addition: three new geometries to verify, not one), and is recorded here so it isn’t silently lost.

3.6 DLVO (electrostatic double-layer + van der Waals)

$$U(r)/k_B T = U_{\text{el}}(r) + U_{\text{vdW}}(r), \quad r > \sigma, \quad (16)$$

$$U_{\text{el}}(r) = \frac{L_B Z^2}{(1 + \kappa a)^2} \frac{e^{-\kappa(r-\sigma)}}{r}, \quad U_{\text{vdW}}(r) = -\frac{A}{6} \left[\frac{2a^2}{r^2 - 4a^2} + 2 \left(\frac{a}{r} \right)^2 + \ln \left(1 - 4 \left(\frac{a}{r} \right)^2 \right) \right], \quad (17)$$

where $a = \sigma/2$, κ the inverse Debye screening length, Z the macro-ion charge, L_B the Bjerrum length, and A the effective Hamaker constant. U_{vdW} 's genuine divergence at $r = \sigma$ is excluded from the computation itself (not merely masked afterward), since the grid places a real point at exactly $r = \sigma$.

3.7 DLVO + hydration repulsion

As Section 3.6, plus a short-range exponential hydration term,

$$U(r)/k_B T = U_{\text{DLVO}}(r) - 2 G_{HY} e^{-(r-\sigma)/D_H}, \quad r > \sigma, \quad (18)$$

with hydration strength G_{HY} and decay length D_H .

3.8 Fermi distribution model (Likos)

$$U(r)/k_B T = \varepsilon \frac{1 + e^{-\sigma/\xi}}{1 + e^{(r-\sigma)/\xi}}. \quad (19)$$

The limit $\xi \rightarrow 0$ reduces to the penetrable sphere model (below), and is handled as a special case when $\xi = 0$ is passed exactly.

3.9 Generalized Gaussian core model (GGCM- n)

$$U(r)/k_B T = \varepsilon \exp \left[-\alpha \left(\frac{r}{\sigma} \right)^n \right]. \quad (20)$$

3.10 Hard sphere + 3 additive Yukawa tails

$$U(r)/k_B T = -\frac{\sigma}{r} \sum_{i=1}^3 K_i \exp \left[-\frac{r-\sigma}{\sigma \lambda_i} \right], \quad r \geq \sigma, \quad (21)$$

with hard core for $r < \sigma$; each (K_i, λ_i) pair may be set to zero to recover fewer effective Yukawa terms.

3.11 Ionic microgel (Likos et al.)

A multi-term analytical potential for a charged microgel sphere of charge Z , dielectric constant ED , and inverse screening length κ_{Π}/σ inside the gel, following Likos et al. (2011); the full piecewise expression (different polynomial/hyperbolic forms inside vs. outside the gel radius) is implemented exactly as in `sasfit_oz_potential_ionic_microgel.c` and is reproduced verbatim in the source rather than restated here, since it spans several auxiliary terms without a single compact closed form.

3.12 Piecewise-constant hard-sphere shoulder/well (3 segments)

$$U(r)/k_B T = \begin{cases} \varepsilon_1 & \sigma \leq r < \sigma + \delta_1 \\ \varepsilon_2 & \sigma + \delta_1 \leq r < \sigma + \delta_1 + \delta_2 \\ \varepsilon_3 & \sigma + \delta_1 + \delta_2 \leq r < \sigma + \delta_1 + \delta_2 + \delta_3 \\ 0 & \text{otherwise (outside all three steps),} \end{cases} \quad (22)$$

hard core for $r < \sigma$; each ε_j may be positive (shoulder) or negative (well). This is the general $n = 3$ piecewise-constant family used as an independent cross-check for the RFA analytical solver (Section 5.7).

3.13 Penetrable sphere model (Likos, Watzlawek, Löwen) [?]

$$U(r)/k_B T = \begin{cases} \varepsilon & r \leq \sigma \\ 0 & r > \sigma. \end{cases} \quad (23)$$

3.14 Soft sphere (inverse power law)

$$U(r)/k_B T = \varepsilon \left(\frac{\sigma}{r} \right)^n. \quad (24)$$

3.15 Parabolic (Hertzian) sphere

$$U(r)/k_B T = \begin{cases} \varepsilon \left(1 - \frac{r}{\sigma} \right)^{5/2} & r \leq \sigma \\ 0 & r > \sigma. \end{cases} \quad (25)$$

3.16 Square well

$$U(r)/k_B T = \begin{cases} \infty & r < \sigma \\ \varepsilon & \sigma \leq r \leq \sigma + \delta \\ 0 & r > \sigma + \delta, \end{cases} \quad (26)$$

with well width δ (well outer range $\lambda = 1 + \delta/\sigma$ in the notation of Section 5.3); $\varepsilon < 0$ for an attractive well, $\varepsilon > 0$ for a repulsive shoulder.

3.17 Sticky hard sphere (Baxter's adhesive limit)

$$U(r)/k_B T = \begin{cases} \infty & r < \sigma \\ \ln \left(\frac{12 \tau \delta}{\sigma + \delta} \right) & \sigma \leq r \leq \sigma + \delta \\ 0 & r > \sigma + \delta, \end{cases} \quad (27)$$

using Baxter's stickiness parameter τ (small τ = sticky, large τ = hard-sphere limit) and a finite well width δ — the same τ convention as SASfit's own `robertus_shs` plugin, giving a natural point of comparison between this (monodisperse, OZ-based) solve and that (polydisperse, PY-only, multicomponent) one.

4 Closure Relations

Every closure below expresses $c(r)$ as a function of $\Gamma(r)$ (and, for the switching-function and Gstar-based closures, the potential's own repulsive/attractive split) and is dispatched through `OZfixpointOperator.update_c()`. Formulas were cross-checked directly against SASfit's own `sasfit_oz_solver.c` case statements (not re-derived from a textbook), using that file's own notation: $\text{EN}(r) = \exp[-U(r)/k_B T]$, $G(r) = \Gamma(r)$. Where a closure needs an extra scalar parameter (shared storage slot `self.alpha`, matching the C code's own `sBPGG/aCJVM/fBB #defines` onto the same underlying variable), this is noted explicitly. **24 closures are offered** – 23 general-purpose ones plus ZSEP (Section 4.20, hard spheres only, self-fitting its own three free parameters rather than taking any of them from the caller); a 25th (Euler–Rahman) exists in the source but is deliberately excluded from both the GUI and `ozLib.solve()`'s validated closure list – see Section 4.21 for why.

4.1 Percus–Yevick (PY)

$$c(r) = \text{EN}(r)[1 + \Gamma(r)] - \Gamma(r) - 1. \quad (28)$$

Equivalent to the standard PY form $c(r) = (e^{-U/k_B T} - 1)y(r)$ with cavity function $y(r) = g(r)e^{U/k_B T}$.

4.2 Hypernetted Chain (HNC)

$$c(r) = \text{EN}(r) e^{\Gamma(r)} - \Gamma(r) - 1. \quad (29)$$

4.3 Reference HNC (RHNC)

Needs a reference-system sub-solve: a bare hard sphere of the same σ is first solved with the Martynov–Sarkisov closure (Section 4.12), giving reference cavity/indirect-correlation functions $g_0(r)$, $\Gamma_0(r)$. Writing the actual potential's perturbation as $\Delta U(r)/k_B T = -\ln \text{EN}(r)$ (zero inside the reference hard core), the closure is

$$c(r) = g_0(r) \exp[(\Gamma(r) - \Gamma_0(r)) - \Delta U(r)/k_B T] - \Gamma(r) - 1. \quad (30)$$

This reduces to the same hard-core exclusion every other hard-sphere closure uses automatically, since $g_0(r) \equiv 0$ inside the reference hard core. Orchestrated by `OZsolver.solveRHNC()`.

4.4 Modified HNC (MHNC)

HNC plus a fixed, purely analytical Percus–Yevick hard-sphere bridge function $B_{\text{PY}}(r; \eta)$ (no reference-system sub-solve needed, unlike RHNC, since B_{PY} has a closed form):

$$c(r) = \text{EN}(r) \exp[\Gamma(r) + B_{\text{PY}}(r; \eta)] - \Gamma(r) - 1, \quad (31)$$

with the packing-fraction-like parameter η supplied by the user (`doMHNCclosure(eta)`).

4.5 Mean Spherical Approximation (MSA)

$$c(r) = \begin{cases} -[\Gamma(r) + 1] & r < \sigma \quad (\text{forces } g(r)=0) \\ -U(r)/k_B T = \ln \text{EN}(r) & r \geq \sigma. \end{cases} \quad (32)$$

4.6 Rescaled MSA (RMSA)

Identical formula to MSA, but the inside/outside switch is driven by an *apparent* hard-core gate $g_{4g}(r) \in \{0, 1\}$ rather than the bare potential's own hard core:

$$c(r) = \begin{cases} -[\Gamma(r) + 1] & g_{4g}(r) = 0 \\ \ln \text{EN}(r) & g_{4g}(r) = 1. \end{cases} \quad (33)$$

`OZsolver.solveRMSA()` first solves plain MSA; if the resulting $g(\sigma^+) < 0$ (an unphysical result plain MSA can give for strongly-coupled systems), it re-solves with the apparent hard core bisected outward, grid point by grid point, until g at the new apparent contact point is non-negative. This is the same Hayter–Penfold-style rescaling idea underlying Section 5.4's semi-analytical RMSA solver, applied here inside the general numerical OZ solve instead of via the closed-form Hayter–Penfold machinery.

4.7 Modified MSA (mMSA)

As MSA, but using the Mayer function outside the core instead of the bare potential:

$$c(r) = \begin{cases} -[\Gamma(r) + 1] & r < \sigma \\ \text{EN}(r) - 1 & r \geq \sigma. \end{cases} \quad (34)$$

4.8 “Symmetric” MSA (SMSA)

Using $\Gamma^*(r) = \Gamma(r) - U_{\text{attr}}(r)/k_B T$ (the attractive-part-subtracted indirect correlation function, shared with several closures below):

$$c(r) = \exp[U_{\text{attr}}(r)/k_B T] [\Gamma^*(r) + 1] - \Gamma(r) - 1. \quad (35)$$

4.9 Rogers–Young (RY) [?]

Interpolates between PY ($\alpha \rightarrow \infty$) and HNC ($\alpha \rightarrow 0$) via a switching function $f(r) = 1 - e^{-\alpha r}$:

$$c(r) = \text{EN}(r) \left[1 + \frac{e^{f(r)\Gamma(r)} - 1}{f(r)} \right] - \Gamma(r) - 1, \quad (36)$$

with α supplied by the user (`doRYclosure(alpha)`).

4.10 HMSA (Zerah–Hansen) [?]

Same switching-function structure as RY, but using $\Gamma^*(r)$ in the switching term and the full Boltzmann factor split into repulsive/attractive parts:

$$c(r) = \exp[-U_{\text{rep}}(r)/k_B T] \left[1 + \frac{e^{f(r)\Gamma^*(r)} - 1}{f(r)} \right] - \Gamma(r) - 1, \quad (37)$$

with α (in $f(r) = 1 - e^{-\alpha r}$) supplied by the user (`doHMSAclosure(alpha)`).

4.11 Verlet [? ?]

Empirical bridge function, no reference system or extra parameter:

$$B(r) = \frac{-\Gamma(r)^2}{2[1 + 0.8\Gamma(r)]}, \quad c(r) = \text{EN}(r) e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \quad (38)$$

4.12 Martynov–Sarkisov (MS) [?]

$$B(r) = \text{sgn}(1+2\Gamma)\sqrt{|1+2\Gamma(r)| - \Gamma(r) - 1}, \quad c(r) = \text{EN}(r) e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \quad (39)$$

Used standalone, and as the reference-system closure for RHNC (Section 4.3).

4.13 Vompe–Martynov (VM) [?]

As MS, but using $\Gamma^*(r)$ in place of $\Gamma(r)$ inside the square root:

$$B(r) = \text{sgn}(1+2\Gamma^*)\sqrt{|1+2\Gamma^*(r)| - \Gamma^*(r) - 1}, \quad c(r) = \text{EN}(r) e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \quad (40)$$

4.14 BPGG (Ballone–Pastore–Galli–Gazzillo) [?]

Power-law bridge function with user parameter s (`doBPGGclosure(s)`, $s \neq 0$; SASfit’s own `sBPGG`):

$$B(r) = \text{sgn}(1+s\Gamma) |1+s\Gamma(r)|^{1/s} - \Gamma(r) - 1, \quad c(r) = \text{EN}(r) e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \quad (41)$$

4.15 CJVM (Charpentier–Jakse extension of VM) [? ?]

User parameter a (`doCJVMclosure(a)`, $a \neq 0$; SASfit’s own `aCJVM`):

$$B(r) = \frac{1}{2a} \left[\text{sgn}(1+4a\Gamma^*) \sqrt{|1+4a\Gamma^*(r)| - 1 - 2a\Gamma^*(r)} \right] - \Gamma^*(r) - 1, \quad (42)$$

$$c(r) = \text{EN}(r) e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \quad (43)$$

4.16 BB (Bomont–Bretonnet) [?]

User parameter f (`doBBclosure(f)`; SASfit’s own `fBB`):

$$B(r) = \text{sgn}(1+2\Gamma^*+f(\Gamma^*)^2) \sqrt{|1+2\Gamma^*(r)+f\Gamma^*(r)^2| - \Gamma^*(r) - 1}, \quad (44)$$

$$c(r) = \text{EN}(r) e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \quad (45)$$

4.17 Kovalenko–Hirata (KH) [?]

$$d(r) = \Gamma(r) - U(r)/k_B T = \Gamma(r) + \ln \text{EN}(r), \quad g(r) = \begin{cases} \text{EN}(r) e^{\Gamma(r)} & d(r) \leq 0 \\ 1 + d(r) & d(r) > 0, \end{cases} \quad (46)$$

$$c(r) = g(r) - \Gamma(r) - 1. \quad (47)$$

This is the real literature formula (equivalently, $b(r) = [\ln(1+d(r)) - d(r)] \theta(d(r))$), matching Kovalenko & Hirata’s own closure [?] as reproduced independently in Pihlajamaa & Janssen’s (2024) closure-comparison table [?] – identical to HNC wherever $d(r) \leq 0$, and linearised (bounded, non-exponential in $\Gamma(r)$) wherever $d(r) > 0$. This is the actual mechanism behind KH’s well-documented good numerical convergence [?]: HNC’s own $c(r)$ grows without bound as $\Gamma(r)$ grows, whereas KH’s linear branch, $c(r) = \ln \text{EN}(r) = -U(r)/k_B T$, does not depend on $\Gamma(r)$ at all, and so cannot diverge that way.

Remark 1. *An earlier version of this port matched SASfit’s own case KH: block in `sasfit_oz_solver.c` exactly, rather than this formula: that C code computes the piecewise-linearised bridge function above too, but only as a separate diagnostic `BRIDGE[i]` output that is never fed back into $c(r)$ in that same closure branch – so that earlier port was, in effect, plain HNC under a different name (confirmed then: identical $g(r)$ to machine precision for every case tested). This*

has now been corrected to the formula above, chosen and verified directly against the literature rather than against that specific quirk of SASfit’s own C source.

Verified three ways. Continuity: the two branches give identical $c(r)$ and $g(r)$ at the switch boundary $d(r) = 0$ (checked to 10^{-10}), so the correction introduces no discontinuity. Hard-core exclusion: where $\text{EN}(r) = 0$, $\ln \text{EN}(r) \rightarrow -\infty$ forces $d(r) \leq 0$, selecting the HNC branch, which reduces to the same $c(r) = -(\Gamma(r) + 1)$ hard-core exclusion PY/HNC/MSA all use – confirmed numerically for **HardSphere**, not just algebraically. Full self-consistent solve: for hard spheres at $\varphi = 0.3$ (Picard iteration, this project’s own solver), the corrected KH gives $g(\sigma^+) = 2.360$, close to the exact PY value (2.356) and clearly different from plain HNC’s own 2.845 at the same state point – expected, since a bare hard-sphere potential has $\text{EN}(r) = 1$ exactly for $r > \sigma$, so KH’s linear branch there gives $c(r) = \ln(1) = 0$, the same value PY’s own formula happens to give outside contact for a hard sphere. At $\varphi = 0.45$, neither the corrected KH, HNC, PY, nor the previous (HNC-identical) KH implementation converges within 2000 plain-Picard iterations – the same already-documented Picard limitation at this density (Section 9.4), not a KH-specific issue: all four closures hit the iteration cap with comparably-sized residuals, so this is not evidence either for or against KH’s own convergence properties specifically; a fair test of that specific literature claim would need an accelerated solver (Section 16), which was not attempted here.

4.18 Duh–Haymet (DH) [?]

A rational-function bridge approximation:

$$B(r) = \frac{-\Gamma^*(r)^2}{2 \left[1 + \frac{5\Gamma^*(r) + 11}{7\Gamma^*(r) + 9} \Gamma^*(r) \right]}, \quad c(r) = \text{EN}(r) e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \quad (48)$$

Independently verified against Pihlajamaa & Janssen (2024) [?], Table 1: $b(r) = -\frac{1}{2}\gamma^2(r)[1 + \gamma(r)(5\gamma(r)+11)/(7\gamma(r)+9)]^{-1}$, an exact match. *Known gap:* SASfit’s own **case DH**: additionally special-cases the Γ^* formula specifically when the active potential is Lennard-Jones (a narrower numerical-stability tweak for that potential’s own r^{-12} term, not a different closure formula); this special case is not replicated here.

4.19 Choudhury–Ghosh (CG) [?]

A piecewise bridge function with a density-dependent coefficient:

$$B(r) = \begin{cases} \frac{-\frac{1}{2}\Gamma^*(r)^2}{1 + \zeta(\varphi)\Gamma^*(r)} & \Gamma^*(r) > 0 \\ -\frac{1}{2}\Gamma^*(r)^2 & \Gamma^*(r) \leq 0, \end{cases} \quad \zeta(\varphi) = 1.0175 - 0.275 \frac{6\varphi}{\pi}, \quad (49)$$

$$c(r) = \text{EN}(r) e^{\Gamma(r)+B(r)} - \Gamma(r) - 1. \quad (50)$$

Independently verified against Pihlajamaa & Janssen (2024) [?], Table 1: $\zeta(\rho) = 1.0175 - 0.275 \rho \sigma^3$, and since $\rho \sigma^3 = 6\varphi/\pi$ for spheres, this is an exact match (including the same $\Gamma^* \leq 0$ piecewise branch).

4.20 ZSEP (Lee) [? ?] — hard spheres only

Unlike every closure above, ZSEP is not a fixed formula the caller parametrises: it self-fits its own three free parameters (ζ, φ_Z, α – **zsep_zeta/zsep_phi/zsep_alpha**, distinct from the system’s own volume fraction φ despite the similar name) against three *exact* conditions derived from the

Carnahan–Starling hard-sphere equation of state, following Lee (1995) [?], Eqs. (2.6)–(2.10) and (3.7)–(3.9). Using $\Gamma^*(r) = \Gamma(r) + \frac{\rho}{2}\text{MAYER}(r)$, $\text{MAYER}(r) = \text{EN}(r) - 1$:

$$B(\Gamma^*) = -\frac{1}{2}\zeta(\Gamma^*)^2 \left(1 - \frac{\varphi_Z \alpha \Gamma^*}{1 + \alpha \Gamma^*}\right), \quad c(r) = \text{EN}(r) e^{\Gamma(r)+B(r)} - \Gamma(r) - 1, \quad (51)$$

with $(\zeta, \varphi_Z, \alpha)$ fixed by requiring, simultaneously:

- (A) the first zero-separation theorem, $B(0) = \beta\mu_{\text{ex}} - \Gamma(0)$, with $\beta\mu_{\text{ex}}$ and $\Gamma(0)$ both closed-form from Carnahan–Starling (the latter needing one correction integral I_C , evaluated numerically here directly from the current solve’s own $g(r)$ and $c(r)$, rather than the original paper’s own reliance on an external correlation for it);
- (B) the second zero-separation theorem, $dB/d\Gamma^*|_0 = 1/y(\sigma^+) - 1$ – **sign-corrected**: the original 1995 paper’s own Eq. (3.8) states the opposite sign; re-deriving this directly from $B(r) = \ln y(r) - \Gamma(r)$ independently confirms the later 1999 erratum’s [?] sign instead, which is what is implemented here;
- (C) pressure consistency (compressibility route = virial route), the same condition Rogers–Young-family closures use (Section 20’s own `findThermodynamicallyConsistentParameter()`), reused here in a three-parameter-aware form.

Matching the validated practice of Pihlajamaa & Janssen (2024) [?] – who report being unable to robustly satisfy all three conditions via exact simultaneous root-finding – this port minimises the sum of squared residuals (`scipy.optimize.least_squares`) rather than attempting an exact 3×3 nonlinear solve.

Hard spheres only, enforced twice. ZSEP’s three conditions are derived specifically from the exact Carnahan–Starling hard-sphere equation of state; they are not meaningful for any other potential. This is checked in two places, not one: `OZsolver.fitZSEPparameters()` itself refuses (prints a warning, returns `None` without solving) if the active potential is not `HardSphere`, and – added specifically because that alone would otherwise let `ozLib.solve()` silently package up an all-zeros/ $S(Q)=1$ -everywhere result with no exception, only an easily-missed Log-tab message – `ozLib.solve()` now raises `ValueError` immediately if `closure='ZSEP'` is requested with any potential other than `'HardSphere'`, before touching the solver at all.

Known, documented ill-conditioning (not a bug). This 3-parameter fit is genuinely under-determined: a whole family of $(\zeta, \varphi_Z, \alpha)$ triples can satisfy the three conditions equally well, and an unconstrained fit can land on a mathematically valid but unphysical-looking point (e.g. very large φ_Z , α near 0) far from Lee’s own reported values – matching Fernaund, Lomba & Lee (2000)’s [?] own explicit finding of multiple solutions to the same optimisation. Fixing $\alpha = 1.0$ (`fitZSEPparameters(fixedParam=('alpha', 1.0))`), Lee’s own empirical finding, Table II of [?]: $\alpha \approx 1.0$ works well at every density from 0.1 to 0.9) and starting near literature values gives good, well-behaved results in practice; fully unconstrained 3-parameter fits should have their output sanity-checked (all three parameters within a physically reasonable $\mathcal{O}(0.1)$ – $\mathcal{O}(10)$ range) before being trusted.

4.21 Euler–Rahman (EuRah) — not offered

Based on Eu & Rah, *J. Chem. Phys.* **111**, 3327 (1999) [?], “A closure for the Ornstein–Zernike relation that gives rise to the thermodynamic consistency.” Unlike every closure above, $c(r)$ here is not a local function of $\Gamma(r)$ alone: it is fixed externally to an array $c_{\text{EuRah}}(r)$ that must itself satisfy an outer self-consistency condition,

$$c_{\text{EuRah}}(r) = \frac{1}{36} [\text{MAYER}(r)] \left[36 y(r) + 18 \varphi \frac{\partial y}{\partial r} + 12 r \frac{\partial y}{\partial \varphi} + 6 \varphi r \frac{\partial^2 y}{\partial r \partial \varphi} \right], \quad (52)$$

where $\text{MAYER}(r) = \text{EN}(r) - 1$, $y(r)$ is the cavity function $y(r) = g(r)e^{U(r)/k_B T}$ evaluated at the trial c_{EuRah} , and the density derivatives are evaluated by re-solving the OZ equation at $\varphi \pm 0.01\varphi$ for the same trial c_{EuRah} . Structurally, writing $36y(r)$ as the dominant term, Eq. 52 is exactly “PY closure plus systematic density/spatial-derivative correction terms” ($c_{\text{PY}} = \text{MAYER} \cdot y$), which is the expected shape for a closure designed to restore thermodynamic consistency to PY.

This closure is implemented (`OZsolver.solveEuRah()`, `doEuRahClosure()`) but is deliberately excluded from `ozLib.CLOSURE_SETTERS` and therefore from both the GUI dropdown and `ozLib.solve()`’s validated closure list. Extensive testing found a genuine exponential positive-feedback instability intrinsic to Eq. 52’s own structure: inside the hard core, $y(r) = e^{\Gamma(r)}$, and $\Gamma(r)$ itself depends on $c_{\text{EuRah}}(r)$ through the OZ equation, so a more negative trial c_{EuRah} produces an exponentially larger $y(r)$, which (via the dominant $36y(r)$ term) produces an even more negative c_{EuRah} on the next iteration. This defeated every numerical strategy attempted: Newton–Krylov (`scipy.optimize.newton_krylov`), damped fixed-point iteration (multiple damping factors), density continuation with warm restarts, dense-Jacobian Newton methods (`fsolve/hybr`, Levenberg–Marquardt, Broyden1/2, `df-sane`), a trust-region-style adaptive step limiter, and SUNDIALS’ own KINSOL (via `sundials4py`) with this project’s best-validated tuning parameters (Section 6.5) — the last of these even produced low-level crashes (seg-fault/bus error) rather than merely failing to converge. The formula’s overall structure was independently verified as plausible (see above), and the literature explicitly notes that this class of “second-order” closure is excluded from standard closure-comparison studies for “numerical complexity” [?]; the instability found here is consistent with that being a genuine property of the theory rather than an implementation error.

4.22 Practical guidance: which closure for which potential

Choosing a closure is a tradeoff between accuracy, generality, and cost (RHNC/RMSA need a reference sub-solve; RY/HMSA/MHNC/BPGG/CJVM/BB need either a manually-chosen parameter or an expensive `findThermodynamicallyConsistentParameter()` search; ZSEP self-fits three parameters by nonlinear least squares). The guidance below reflects standard liquid-state-theory practice together with what has actually been validated in this project (Sections 9, 5) – it is a starting point for exploration, not a guarantee that any one closure is quantitatively correct for a given system without checking against an independent reference (Section 5) where one is available.

- **Hard spheres, or any purely repulsive, short-ranged potential** (HardSphere, SoftSphere, GGCM-n, PenetrableSphere): start with **PY**. It is simple, cheap, and (Section 9’s own validation) matches the exact hard-sphere solution essentially exactly at the densities tested here – though see Section 4.23 below: against genuine Monte Carlo data (not the exact PY solution itself), PY’s own contact-value error is not actually small at high density [?]. **ZSEP** (Section 4.20) is a genuinely more accurate option specifically for HardSphere (only), if its known ill-conditioning (fix $\alpha = 1$, check the fitted parameters look physically reasonable) is acceptable for the use case.
- **Attractive or narrow-well potentials** (StickyHardSphere, SquareWell, LennardJones, PiecewiseConstant): PY tends to be least accurate here (Section 9’s own closure comparison shows PY and HNC bracketing the true contact value from opposite sides at high density, a general feature that sharpens with attraction, confirmed quantitatively against Monte Carlo in Section 4.23 [?]); an empirical bridge-function closure (**Verlet**, **MS**, **VM**, **DH**, **CG**) or **HNC** itself is generally a better default. **HMSA** (needs a manually chosen or fitted α) is specifically designed for exactly this case – a potential with a genuine attractive tail split from a repulsive core – since it uses that split directly (Section 20’s own HMSA formula). For narrow/strong wells specifically, cross-check against the semi-analytical references built for exactly this case (Sharma–Sharma, ORPA, RFA, Section 5)

if available for the potential in use, since this project’s own worked examples show a naive perturbative treatment (Sharma–Sharma) can degrade substantially for strong/wide wells where a genuinely non-perturbative one (ORPA) does not. For **StickyHardSphere** specifically, published Monte Carlo comparisons [?] find plain PY performs adequately in the parameter regime typical of colloidal/protein solutions, but is out-performed by more sophisticated closures elsewhere – see Section 4.23.

- **Long-range, charged, or screened-Coulomb potentials** (DLVO, DLVO Hydra, HS3Yukawa, IonicMicrogel): **MSA** is the standard, cheap starting point, but can give an unphysical negative contact value at strong coupling (Section 9.3); **RMSA** is the direct fix for exactly that failure mode and should be preferred whenever MSA’s own contact value comes out negative. Where the closed-form Hayter–Penfold solution applies (a bare hard-core-Yukawa potential, Section 5.4), that semi-analytical reference is both faster and, being the historically canonical solution for this exact problem, a useful independent check on the general numerical solver’s own RMSA rescaling. **HNC**, by contrast, is well-documented in the plasma/charged-colloid literature to degrade, and in some regimes fail to converge at all, at strong coupling (Section 4.23) – this is specifically why Rosenfeld’s variational modified HNC (VMHNC) [?] and similar corrections exist; **BPGG** (already offered here, and itself one of the corrections used in that literature) is a reasonable substitute for plain HNC at strong coupling if MSA/RMSA is not applicable.
- **When nothing is known about the system: Verlet.** This is the headline recommendation of [?], whose comparison across hard-sphere, inverse-power-law, Gaussian-core and Lennard-Jones systems in both two and three dimensions found Verlet “robustly performed well for all our benchmark systems, indicating that it could suitably serve as a first guess for use in unfamiliar systems”. It has no free parameter, so it costs no consistency search. Their central negative finding is worth stating alongside it: no closure performed best across all systems, so the entries in this list are starting points and not a ranking.
- **Among the consistency-parameter closures, split by potential softness:** the same study found **Modified Verlet** and **CJVM** (Charpentier–Jakse) both excellent in most cases, with MV more accurate for *steeply repulsive* potentials and CJ superior for *softer, longer-ranged* ones. Both are offered here (Sections 20) and both take the consistency search. This is the most directly actionable recommendation in that work for anyone fitting colloidal data, and neither closure is otherwise singled out in this section.
A caveat that matters for this software specifically: their comparison covers single-component, uncharged, spherically symmetric systems. The potentials most used here — short-range attractive with long-range repulsion, square wells, charged colloids — are exactly the ones they exclude, as “such systems often depend on a multitude of parameters that must be carefully chosen”. So the split above is a starting point for those cases rather than a conclusion, which is an argument for running the approximation-error panel on the system at hand.
- **When thermodynamic consistency matters most** (equation-of-state or virial-route quantities, not just $g(r)/S(Q)$ shape): **Rogers–Young** (or **HMSA** for potentials with a genuine attractive tail) with `findConsistentParameter=True` is specifically designed to enforce this by construction, at the cost of the extra root-finding search (Section 7). **MHNC**, **BPGG**, **CJVM**, and **BB** offer the same consistency search (`CONSISTENT_PARAMETER_CLOSURES`) as alternative bridge-function forms, worth trying if Rogers–Young/HMSA’s own search fails to find a genuine root for a particular system.
- **Highest achievable general accuracy, cost secondary:** **RHNC** (a genuine reference-system correction, no free parameter to choose) or **MHNC** (a fixed analytical PY bridge function, cheaper than RHNC since no reference sub-solve is needed) are the more sophisticated options among what is offered here – consistent with Section 4.23’s own finding that MHNC-type closures (parameterised directly from simulation data) systematically

outperform parameter-free closures against genuine Monte Carlo benchmarks [?].

- **Avoid: Euler–Rahman** (Section 4.21) – excluded from the GUI/ozLib.solve() entirely for a genuine, documented convergence instability, not merely a missing feature. The hand-written **Anderson acceleration** solver (Section 16, not a closure but worth flagging here too) should not be trusted uncross-checked on strongly-coupled/charged systems specifically, having been found to converge silently to a wrong answer there in this project’s own testing (Section 9.4).
- **Kovalenko–Hirata**: now implements the real literature formula (Section 4.17), not the SASfit-C-code quirk an earlier version of this port had (which was, in effect, plain HNC under a different name). Its main practical advantage over plain HNC is bounded, non-exponential behaviour when $\Gamma(r)$ grows large, per the literature [?] – worth trying wherever HNC itself is a reasonable choice (Section 4.23) but its own exponential growth is a concern.
- **Khanpour**: a one-parameter softening of HNC, $B = \ln(1 + \alpha\Gamma)/\alpha - \Gamma$, recovering HNC exactly as $\alpha \rightarrow 0$. Its bridge function grows only logarithmically in Γ , so like Kovalenko–Hirata it is worth trying wherever HNC is a reasonable choice but its exponential growth is a concern – with the advantage over KH that the single parameter can be fixed by the consistency search (CONSISTENT_PARAMETER_CLOSURES). Not for strongly charged systems: on the polydisperse charged Yukawa test case it returns $g_{NN} \approx 1276$, the same qualitative failure MSA (2484) and PY (292) show there.
- **Modified Verlet**: Verlet’s bridge function with the denominator switched off for $\Gamma < 0$, $B = -\Gamma^2/2$ there and $-(\Gamma^2/2)/(1+\alpha\Gamma/2)$ otherwise. Same domain of applicability as plain Verlet – attractive and narrow-well potentials – and worth trying as an alternative when Verlet’s own denominator approaches zero. Its parameter is also consistency-searchable.
- **Extended Rogers–Young**: the Rogers–Young construction with one extra quadratic term inside the logarithm, so that $a = 0$ reduces to RY *exactly* (verified here to 2.5×10^{-16}). Its value is precisely that second degree of freedom: where RY’s own consistency search finds no root – which happens, since the residual is often monotone and already nonzero in the PY limit (Section 15.2) – the extra parameter gives the search somewhere else to go. Use it as the first fallback when Rogers–Young cannot be made consistent for a given system.
- **Carbajal–Tinoco** [?]: the bridge function of the 2022 paper on local thermodynamic consistency, in which λ interpolates between known closures — $\lambda = 0$ recovers Martynov–Sarkisov and $\lambda \rightarrow \infty$ recovers HNC. The amplitude is r -dependent for $\lambda \leq 0$ and constant above.

It is the only *implicit* closure in the library: its bridge function must be solved for at every grid point, which is done by a vectorised fixed-point iteration rather than a scalar root find per point.

It has a bounded domain, and outside it there is no solution to find. Substituting $w = \Gamma + b$ turns the implicit relation $b = T(\Gamma + b)$ into the explicit map $\Gamma(w) = w - T(w)$, whose range is bounded below: $\Gamma_{\min} = -0.509$ at $\lambda = 0$, rising towards zero as λ grows, with two solutions above it and none below. That, and not a repulsive fixed point, is why $\lambda > 0$ fails — it shrinks the admissible region. No iteration can find a root that does not exist, and damped Picard, Newton, and Newton with a backtracking line search were each tried. Neither the source paper nor the comparison of [?] locates that boundary; the latter simply excludes closures from its figures for convergence issues.

The current code iterates 500 damped passes and returns the last one whether or not it converged, so a solve whose path crosses the boundary does so silently. The reference implementation raises instead, which is the better behaviour. Within the domain it reproduces that reference to 2×10^{-13} .

An earlier version of this section attributed the closure to Carbajal–Tinoco’s 2008 paper

on soft repulsive spheres. That paper is the source of **Extended Rogers–Young**, implemented separately here; the misattribution came from *OrnsteinZernike.jl*’s source docstring, which contradicts Table I of its own accompanying paper, and was inherited along with the implementation.

- **For polydisperse systems**, the choice narrows on structural grounds rather than accuracy: 20 of the 24 closures work multicomponent unchanged, because their bridge functions are elementwise in Γ . The four that do not — **Reference HNC** and **Modified HNC** (both need a one-component reference solve), **Rescaled MSA** (a one-component rescaling procedure) and **EuRah** — are not offered at all once a polydisperse potential is selected, rather than being offered and failing. Note in particular that RMSA’s usual role as the fix for MSA’s negative contact value is therefore *not* available on the numerical polydisperse route; the analytic `polydisperse_rmsa` module remains the way to get it. **MS**, **VM** and **CJVM** are offered but should be expected to fail on strongly coupled charged systems: all three build a square-root bridge function and $\exp(\Gamma + B)$ overflows there — a closure-domain limit that applies equally in one component.

4.23 Literature validation against Monte Carlo: known accurate and inaccurate combinations

The recommendations above draw on published, quantitative comparisons against Monte Carlo (MC) or molecular dynamics (MD) simulation – the genuine ground truth for a given potential, as opposed to a closure’s own internal thermodynamic consistency, which (Section 20’s own introduction) can be satisfied without the correlation functions themselves being accurate. This section summarises what is actually documented in the literature for the specific potential/closure pairs offered here, including where combinations are *known to give poor results*, not only where they succeed – since a plugin’s own thermodynamic self-consistency check passing is not, by itself, evidence of accuracy against MC.

Hard spheres. Pihlajamaa & Janssen (2024) [?] performed a systematic MC comparison across many closures at $\rho\sigma^3 = 0.94$ (just below melting) and report the PY and HNC closures as the two *worst*-performing of every closure they tested, PY underestimating and HNC overestimating the contact value: $g_{PY}(\sigma) - g_{MC}(\sigma) = -0.921$, $g_{HNC}(\sigma) - g_{MC}(\sigma) = +1.966$ – confirming the long-known qualitative result that PY outperforms HNC for hard spheres, but quantifying just how large both errors are at this density. Across every system they tested (hard spheres, inverse-power-law, Gaussian-core, and Lennard-Jones, in two and three dimensions), their overall conclusion is that “the well-known Percus-Yevick equation performed worse than any among the V[erlet], DH, and MS closures” – i.e. PY’s continued popularity reflects familiarity and ease of implementation, not superior accuracy, a point their paper makes explicitly. The Lee/ZSEP closure specifically (Section 4.20) was found to give a *better* contact value than the other thermodynamically-consistent closures they tested, which the authors attribute directly to its built-in use of the Carnahan–Starling equation of state – independent confirmation, from outside this project, of exactly the ZSEP behaviour already documented in Section 4.20.

Sticky/adhesive hard spheres. Gazzillo & Giacometti (2004) [?] solved Baxter’s sticky-hard-sphere model analytically for several closures (including PY) and compared directly against MC. Their finding is a genuine middle-of-the-road result for PY, not a clear pass or fail: PY’s predictions “lie between simpler closures, which may yield less accurate predictions but are easily extensible to multicomponent fluids, and more sophisticat[ed] closures which give more precise predictions but can hardly be extended to mixtures.” They further report that “in regimes typical for colloidal and protein solutions,” first-order perturbative closures (including PY) “produce satisfactory results” – i.e. PY’s adequacy for sticky spheres is regime-dependent, not universal, consistent with this project’s own choice to offer the Baxter/PY solution (Section 5) as a validated reference specifically for that regime rather than as a claim of general

accuracy.

Lennard-Jones. The same Pihlajamaa & Janssen study [?] finds that renormalising the indirect correlation function ($\gamma^*(r) = \gamma(r) - \beta u_{\text{LR}}(r)$, the same attractive/repulsive potential split this project’s own HMSA/RYS closures use, Section 3) is necessary for good accuracy: plain PY applied without this renormalisation is markedly less accurate than the renormalised closures (CG*, Verlet*, DH*) at their tested state point ($\rho\sigma^3 = 0.8$, $k_B T/\varepsilon = 1.0$, peak $g(r_{\text{peak}}) \approx 2.6$).

Charged and screened-Coulomb (Yukawa) systems. Unlike the short-ranged cases above, the literature on strongly-coupled Yukawa/one component-plasma systems documents outright *failure* of plain HNC, not merely reduced accuracy: HNC is known not to have a solution at all in certain regions of the phase diagram approaching phase separation for hard-core-Yukawa and related charged systems, with the boundary of non-existence not coinciding with the true spinodal. This is the direct motivation for Rosenfeld’s variational modified HNC (VMHNC) [?] and later isomorph-based modified-HNC variants, both explicitly constructed and validated against MD specifically because plain HNC’s accuracy (and, in some regimes, its very convergence) degrades as coupling strength increases for these systems – directly analogous to, and part of the same broader literature as, the MSA contact-value failure this project’s own RMSA rescaling addresses (Section 4.6) for the closure actually offered here. **BPGG** (Section 20), one of the closures used in that same strongly-coupled-Yukawa literature as an HNC alternative, is offered directly in this toolkit and is a reasonable next step if RMSA is not applicable to the potential at hand.

Scope of what is, and is not, independently confirmed here: the findings above are drawn directly from the cited papers’ own stated results, not re-derived or re-run by this project. Where a cited paper studies a closely related but not identical system (e.g. the one-component-plasma/Yukawa HNC-convergence literature, which concerns bulk charged fluids rather than this project’s own colloidal DLVO/HS3Yukawa potentials specifically), this is noted as such above rather than presented as a direct one-to-one validation of this project’s own numerical results.

5 Analytical and Semi-Analytical Cross-Validation Tools

Independent of the general numerical OZ solve, several closed-form or semi-analytical structure-factor solutions are implemented as cross-checks. These are consulted throughout this project to validate the numerical solvers and closures above, rather than offered as alternative closures themselves.

5.1 Percus–Yevick hard sphere (Wertheim–Thiele)

The classic closed-form PY hard-sphere solution (Naegele’s notation),

$$S(q) = \frac{1}{X(q)^2 + Y(q)^2}, \quad X = 1 - 12\varphi(Af_1 + Bf_2), \quad Y = -12\varphi(Af_3 + Bf_4), \quad (53)$$

with $A = (1 + 2\varphi)/(1 - \varphi)^2$, $B = (1 + \varphi/2)/(1 - \varphi)^2$, and $f_1, \dots, f_4$ standard trigonometric combinations of $y = q\sigma$. Verified directly against SASfit’s own `GPY()` formula; found the existing Python implementation to be numerically *more* robust than SASfit’s own C code at very low q (SASfit’s small-argument Taylor threshold is far tighter than where the raw trigonometric formula actually loses precision).

5.2 Baxter sticky hard sphere (idealised and finite-well)

Baxter’s (1968) closed-form adhesive-sphere solution using stickiness parameter τ , ported directly from `sasfit_sq_sticky_hard_sphere.c`, plus a finite-well generalisation (`sasfit_sq_sticky_hard_sphere_2.c`) for a well of finite width δ — the physically correct comparison for this project’s own finite- δ `StickyHardSphere` potential (Section 3.17), since the idealised $\delta \rightarrow 0$ limit is not the same physical system.

5.3 Sharma–Sharma square well

The classic first-order perturbative PY square-well solution (Sharma & Sharma, *Physica A* **89**, 213 (1977)), ported from `sasfit_sq_square_well_potential.c`:

$$S(q) = \frac{1}{1 - C(q)}, \quad C(q) = \frac{-24\varphi}{\kappa^6} \left[\alpha \kappa^3 (\sin \kappa - \kappa \cos \kappa) + \beta \kappa^2 (2\kappa \sin \kappa - (\kappa^2 - 2) \cos \kappa - 2) \right. \\ \left. + \gamma ((4\kappa^3 - 24\kappa) \sin \kappa - (\kappa^4 - 12\kappa^2 + 24) \cos \kappa + 24) - \varepsilon \kappa^3 (\sin \lambda \kappa - \lambda \kappa \cos \lambda \kappa + \kappa \cos \kappa - \sin \kappa) \right], \quad (54)$$

with $\kappa = q\sigma$, well outer range $\lambda = 1 + \delta/\sigma$, and α, β, γ the same PY-like density coefficients as the hard-sphere solution above. Being a first-order expansion, this closed-form solution degrades for strong/wide wells (verified: max. deviation from the full numerical solve rose from ~ 0.005 at $\varepsilon = -0.02, \delta = 0.02\sigma$ to ~ 0.33 at $\varepsilon = -0.5, \delta = 0.3\sigma$, at $\varphi = 0.15$), which is the expected behaviour of a perturbative expansion approaching the edge of its own validity.

5.4 Rescaled MSA (Hayter–Penfold)

The full Hayter–Penfold RMSA solution for a hard-core Yukawa/screened-Coulomb potential, compiled as a standalone C library (`rmsa_c_source/`, from `src/plugins/RMSA/` – `rmsa.c`, `polyroots.c`, `rmsa_physical.c`) and wrapped via `ctypes` (`rmsaWrapper.py`), reusing SASfit’s own quartic-root selection and small- κ Taylor-series logic rather than re-implementing it. Validated: the weak-coupling limit reproduces the exact PY hard-sphere contact value $g(\sigma^+) = (1 + \varphi/2)/(1 - \varphi)^2$.

5.5 One-Yukawa MSA (Blum–Høye)

A semi-analytical Yukawa-MSA solution, compiled as a standalone C library (`oneyukawa_c_source/`, from `src/plugins/twoyukawa/`) including a full Jenkins–Traub complex polynomial root-finder (`2Y_cpoly.c`) and an FFT-based pair-correlation-function evaluation (`2Y_PairCorrelation.c`) used to select the physically correct root among up to four candidates. Cross-validated against the general numerical OZ solve (HS3Yukawa potential + MSA closure) to within $\sim 2\%$ mean / $\sim 5\%$ max deviation, concentrated at low q and shrinking rapidly at higher q — the same order of residual found for the exact PY hard-sphere case, consistent with a numerical-grid-vs-exact discrepancy rather than an error in either implementation.

5.6 ORPA (Pini–Parola–Reatto) for square wells

A self-consistent optimised random-phase approximation specific to square-well potentials (Pini, Parola & Reatto, *Mol. Phys.* **100**, 1507 (2002)), compiled as a standalone C library (`orpa_c_source/`) using GSL’s derivative-free multivariate root solver with continuation in the well depth. At a strong-well test case ($\varphi = 0.15$, $\varepsilon = -0.5k_B T$, $\delta = 0.3\sigma$) where the Sharma–Sharma perturbative formula (Section 5.3) showed max. deviation 0.33 from the numerical solve, ORPA gave max. deviation 0.057 — roughly $6\times$ closer, consistent with ORPA being a genuinely non-perturbative theory.

5.7 RFA (Rational Function Approximation)

Santos, Yuste & López de Haro’s rational-function approximation for *arbitrary* piecewise-constant potentials (*Condens. Matter Phys.* **15**, 23602 (2012)), compiled as a standalone C library (`rfa_c_source/`) with a numerically-safe small- q series expansion. Because it handles an arbitrary number of piecewise-constant steps, this is the only analytical tool validated against both the single-well `SquareWell` potential ($n = 1$; max. deviation 0.025 on the same strong-well case above, the closest of the three square-well cross-checks) *and* the genuinely multi-step `PiecewiseConstantHS` potential ($n = 3$ shoulder-well-shoulder; max. deviation 0.0056/mean 0.0028), a cross-check no other tool here can perform.

6 Numerical Solver Algorithms

Given a closure (Section 20), the OZ equation (Eq. 2) reduces the physics problem to a fixed-point problem $\Gamma = \mathcal{F}(\Gamma)$, where $\mathcal{F}$ composes the closure and the Hankel-transform-based OZ update. Eight solver algorithms are available, offering a range of speed/reliability tradeoffs (Section 6.7 compares them empirically).

6.1 Picard iteration

Plain, undamped fixed-point iteration, $\Gamma_{n+1} = \mathcal{F}(\Gamma_n)$, until the relative change in $\|\Gamma_n\|$ falls below a convergence criterion. Simple and reliable for easy problems; can fail to converge for harder ones (e.g. dense hard spheres, strongly-charged systems).

6.2 Anderson acceleration (hand-written)

A direct Python implementation of Anderson mixing: the next iterate is a linear combination of the last m residuals, with mixing coefficients chosen by a small least-squares problem, extrapolating the fixed-point iteration rather than taking it verbatim. Confirmed independently reliable in most cases, but found (Section 6.7) to occasionally converge to a spurious answer on very hard problems (strongly-charged MSA) without raising any error — silently disagreeing with cross-validated Newton–Krylov solutions by a wide margin in that regime.

6.3 scipy Anderson / scipy Newton–Krylov

Thin wrappers around `scipy.optimize.anderson/newton_krylov` respectively, applied to $\mathcal{F}$. Reliable across the full range of problems tested here, though can be markedly slower than the alternatives below on both easy and hard problems.

6.4 Biggs–Andrews

A hand-written acceleration scheme, ported directly from `sasfit_oz_solver.c`’s own `CASE BIGGS_ANDREWS`: block (dispatched there through the same `OZ_solver_by_iteration()` as Picard — i.e. this is *not* SUNDIALS-based, despite the KINSOL-adjacent naming of its own tuning parameter). After two unaccelerated warm-up steps, each further step:

1. computes a Barzilai–Borwein-style spectral step size $\alpha = \beta/\gamma$ (or $\text{sgn}(\beta/\gamma)\sqrt{|\beta/\gamma|}$), with $\beta = \langle g_n, g_{n-1} \rangle$, $\gamma = \langle g_{n-1}, g_{n-1} \rangle$ inner products of consecutive fixed-point residuals $g_k = \mathcal{F}(x_k) - x_k$;
2. clamps α to the Nesterov momentum bound $[-\frac{n-1}{n+2}, \frac{n-1}{n+2}]$;
3. extrapolates using order `|KINSetMAA|` (0: none; 1: heavy-ball momentum $y = x + \alpha(x - x_{\text{prev}})$; ≥ 2 : adds a second-order term $\frac{\alpha^2}{2}(x - 2x_{\text{prev}} + x_{\text{prevprev}})$).

Validated: order 0 reproduces plain Picard almost exactly (354 vs. 353 steps on hard-sphere/PY at $\varphi = 0.3$); orders 1/−2 roughly halve the iteration count on that same case; every configuration agrees with the independently validated GMRES solver to $\sim 10^{-12}$.

6.5 SUNDIALS KINSOL: Newton–Krylov (GMRES/FGMRES/TFQMR)

A genuine Newton–Krylov method via SUNDIALS’ own KINSOL, through its official Python bindings (`sundials4py`), using the `KIN_LINESEARCH` strategy with a selectable linear solver. Tuning parameters were matched exactly to SASfit’s own “configure OZ solver” dialog defaults (shared directly by a user of this project): `KINSetMaxNewtonStep=100N` (not just N), `KINSetFuncNormTol=10−10`, `KINSetScaledStepTol=10−13`, `KINSetMaxRestarts=10` (GMRES/FGMRES only), `KINSetEtaForm=KIN_ETACHOICE1`. This made a confirmed, material

difference: a strongly-charged DLVO+MSA case that failed to converge under more conservative defaults converges cleanly under these settings. `SUNLinSol_SPBCGS` (Bi-CGSTAB) is deliberately not offered: it segfaults unconditionally in the tested `sundials4py` version, confirmed in total isolation from any OZ-specific code, i.e. a bug in that package rather than a usage error here.

6.6 SUNDIALS KINSOL: Fixed-Point (Anderson), `KIN_FP`

SUNDIALS’ own Anderson-accelerated fixed-point strategy. This needed a genuinely different `sysfn` convention than the Newton–Krylov solver above — found, by reading `OZ_step_kinsolFP()` in the C source directly, that `sysfn` must return $\mathcal{F}(x)$ itself, not the residual $\mathcal{F}(x) - x$ every other KINSOL-based solver here uses. Once matched, no Anderson damping (`KINSetDampingAA`) was needed, matching SASfit’s own configuration function exactly (which never sets it either); validated to 10^{-12} against the Newton–Krylov solver on two different potentials.

6.7 Empirical solver comparison

Case	Picard	Anderson	scipy NK	Biggs–Andrews	sundials4py GM- RES/KIN_ FP
Easy (HS+PY)	OK	OK (fastest)	OK	OK	OK (fastest)
Moderate (Sticky+MSA)	OK	OK	OK	OK	OK
Dense HS ($\varphi=0.45$)	fail	OK	OK	OK (order ≥ 1 only)	OK
Strongly-charged MSA	fail	<i>wrong answer, uncaught</i>	OK	fail	GMRES: OK; KIN_

Table 1: Reliability across a range of difficulties (see the project history for full timing data). No single solver dominates; `sundials4py` GMRES/FGMRES was the only combination to succeed on every case tried. The hand-written Anderson solver’s silent wrong-answer failure mode on the hardest case is a more concerning outcome than an honest non-convergence report.

7 The ozLib Python Library

ozLib.py is the single source of truth for the calculation workflow — both oZgui.py’s own “calculate” button and any user script call the same ozLib.solve() function, so results from the GUI and from scripts are identical by construction, not by two independently-maintained implementations happening to agree.

7.1 Basic usage

```
from ozLib import solve

result = solve(potential='HardSphere', phi=0.3,
               closure='Percus-Yevick',
               solver='sundials4py: Newton-Krylov (GMRES)')

result.gr # g(r)
result.Sq # S(q)
result.cr # c(r)
result.gamma # Gamma(r)
result.hr # h(r)
result.Ur # U(r)/kT
result.Br # B(r) (bridge function)
result.yr # y(r) (cavity function)
result.fr # f(r) (Mayer function)
result.r, result.q # the real-/reciprocal-space grids
```

7.2 Function signature

```
def solve(potential, phi, potentialArgs=(), closure="Percus-Yevick",
          closureParam=None, findConsistentParameter=False,
          solver=_defaultSolverName, maxIterations=1000,
          numberOfRadialSamplingPoints=None, hardSphereDiameterInPoints=None,
          onSolverCreated=None):
    ...
    return OZResult(...)
```

potential, closure, and solver are validated against ozLib.SOLVER_CLASSES/ozLib.CLOSURE_SETTERS (the canonical, current option lists — print them from a script to see exactly what is available); an unrecognised name, or a closure needing closureParam without one given, raises ValueError immediately rather than failing confusingly deeper inside the solve. numberOfRadialSamplingPoints/ hardSphereDiameterInPoints expose the grid configurability of Section 2 directly. solver’s default, _defaultSolverName, is *not* "Picard iteration" (an earlier version of this document, and an earlier version of the code itself, both had it as the default) — it is now "sundials4py: Fixed-Point (Anderson)" when the optional sundials4py dependency is installed, falling back to "scipy Anderson" otherwise; see ozLib.py’s own comment on SOLVER_CLASSES for why (SUNDIALS’ production-grade Anderson-accelerated fixed-point strategy generally converges more reliably and faster than plain unaccelerated Picard for the harder closures — Section 6.7 quantifies this). Plain Picard iteration remains fully available (solver="Picard iteration"), simply no longer pre-selected. findConsistentParameter: for closures with a free parameter that also have a published thermodynamic-consistency condition determining it automatically (Rogers–Young, HMSA, Modified HNC, BPGG, CJVM, BB — see ozLib.CONSISTENT_PARAMETER_CLOSURES), setting this to True searches for that value instead of requiring closureParam from the caller, via OZsolver.findThermodynamicallyConsistentParameter() (Rogers & Young’s own

compressibility-route/virial-route consistency idea, generalised across potentials rather than limited to any one). Raises `ValueError` if requested for a closure that doesn't support it.

7.3 Example: potential with arguments and a parametrised closure

```
result = solve(potential='StickyHardSphere', phi=0.2,
               potentialArgs=(0.3, 0.05), # (tau, delta)
               closure='Rogers-Young', closureParam=2.0,
               solver='scipy Newton-Krylov')
```

7.4 Example: long-range potential, extended grid

```
result = solve(potential='HS3Yukawa', phi=0.15,
               potentialArgs=(-0.3, 10.0, 0, 1, 0, 1), # (K1, lambda1, ...)
               closure='MSA',
               numberOfRadialSamplingPoints=4*4096, # 4x extended range
               hardSphereDiameterInPoints=100) # same resolution
```

8 The ozGUI Application

`oZgui.py` is a Tkinter application, run directly with `python oZgui.py`, providing:

- **Left panel:** potential dropdown (with dynamically-generated parameter fields, discovered by introspection — a new potential added to `oZfixpointOperator.py` appears automatically, nothing else to edit), closure dropdown (with a parameter field where needed, plus a “find thermodynamically consistent value” checkbox for closures with a published consistency condition, see `CONSISTENT_PARAMETER_CLOSURES` in Section 7, which disables — rather than hides — the manual entry field while checked), volume fraction, solver dropdown, X-axis scale selector (`symlog/linear/log/asinh`, with an adjustable linear-region threshold), grid size controls (`Grid points (N)/ Points per  $\sigma$` , both defaulting to “auto” to keep this project’s own historical grid, Section 2; either can be overridden independently, e.g. to extend the range for a long-range potential without losing near-contact resolution), max. iterations, calculate/interrupt/clear/delete-last buttons, Save/Load (`.oz` JSON format, round-trips exactly including NaN entries in $B(r)/y(r)$), and a single format-selector export control (ASCII/CSV/Excel/PNG-all-tabs/copy- to-clipboard) for the currently selected (or most recent) run.
- **Right panel:** a 10-tab notebook — the same 9 plot tabs as the original Tcl GUI ($S(Q)$, $g(r)$, $c(r)$, $\Gamma(r)$, $h(r)$, $U(r)/kT$, $B(r)$, $y(r)$, $f(r)$), overlaying every run in the history list in a distinct colour, plus a **Log tab** with no Tcl-side equivalent: every status update and every `print()` anywhere in the whole process (including diagnostic output already buried inside `oZfixpointOperator.py/oZsolver.py`, e.g. `fitZSEPparameters()`’s or `findThermodynamicallyConsistentParameter()`’s own messages) is captured here via a thread-safe `stdout/stderr` redirector, timestamped, and tee’d to the real console so nothing is lost even when run windowed (no terminal at all).

Exported ASCII/CSV/Excel data always contains the full-precision computed values regardless of the current plot’s axis scale (verified directly: two exports taken with different X-axis scale/threshold settings in between are byte-identical); only the PNG export reflects the currently displayed scale, since it is a rendered image.

8.1 Relationship to SASfit’s own Tcl GUI, and naming-convention differences

`oZgui.py` is a direct, from-scratch port of SASfit’s own built-in Ornstein–Zernike solver window, `sasfit.vfs/lib/app-sasfit/tcl/sasfit_oZ_solver.tcl` — the same potential/closure/al-

gorithm choices, the same 9 output curves, laid out as a Tkinter equivalent of that Tcl/Tk/BLT window rather than embedded inside SASfit itself. Three things were deliberately *not* carried over, since they either depend on SASfit’s own fitting engine directly or are already covered by a Tkinter/matplotlib equivalent: the “assign to SQ plugin” mechanism (replaced by the Save/Load/export controls of Section 8 instead), the detailed KINSOL tuning sub-dialog (see Table 5 below), and crosshair coordinate readout / zoom-stack (matplotlib’s own toolbar under each plot covers zoom/pan/save already).

Cross-referencing the Tcl source directly against `oZgui.py` / `ozLib.py` (rather than assuming a 1:1 naming correspondence) surfaced a number of places where the same concept is named differently on the two sides, summarised in Tables 2–6 below. The general pattern: Tcl consistently favours short abbreviations matching the underlying SUNDIALS/mathematical-literature names exactly (`RMSA`, `mMSA`, `MS`, `DH`, `KINSetMAA`); this Python port consistently favours spelled-out, more self-explanatory names for anything user-facing (`Rescaled MSA`, `Modified MSA`, `Martynov-Sarkisov`, `Duh-Haymet`), while keeping terse internal dict keys (`gr`, `cr`, `Sq`, ...) at the data-storage layer — i.e. the renaming happens specifically at the human-facing label layer, not throughout.

Tcl OZ(potential) value	Python potential name
HardSphere, StickyHardSphere, SquareWell,	same
PiecewiseConstant, SoftSphere, Fermi,	same
LennardJones, IonicMicrogel, PenetrableSphere,	same
DLVO, DLVO Hydra, GGCM-n, HS 3Yukawa	same
StarPolymer (f>10)	StarPolymerHighF (renamed, not identical)
StarPolymer (f<10)	StarPolymerLowF (renamed, not identical – previously missing entirely, Section 3.4)
Depl-Sph-Sph	
Depl-Sph-Discs	<i>not currently offered</i>
Depl-Sph-Rods	(Section 3.5’s Known gap)

Table 2: Potential-name naming convention. Most match exactly; star polymer now matches in *substance* (both formulas present) but not exact spelling, since Python identifiers can’t contain spaces/parentheses/ inequality signs; the three depletion geometry variants remain a known, unfixed gap. Python’s own list is generated dynamically via introspection of `oZfixpointOperator.py`’s `set<Name>Potential()` methods, so it automatically includes any future addition there; Tcl’s is a fixed, hand-written `-values {...}` string that must be edited by hand.

Not independently re-verified in this pass: the exact Python argument names of `oZfixpointOperator.py`’s `set<Name>Potential()` methods were not individually confirmed character-for-character against the Tcl GUI’s own hand-written parameter display labels for every one of the 18 potentials — most match directly (e.g. `diameter`, `tau`, `delta`, `epsilon`), but three potentials (`SoftSphere`’s “stiffness *n*”, and the two `Depl-Sph-*` potentials’ “diam. (large)”/“diam. (small)”/“*n* (numb. dens.)” style labels) have Tcl-side display strings that are abbreviated prose rather than bare identifiers, making them the most likely spots for the actual Python argument’s spelling to differ from what the Tcl label alone would suggest.

9 Worked Examples: Numerical vs. Analytical Validation

This section collects representative comparisons between the numerical solver and the analytical/semi-analytical references of Section 5, plus the empirical behaviour of the toolkit at difficult and physically interesting parameter combinations.

Tcl OZ(closure) value	Python closure name	Relationship
Percus-Yevick	Percus-Yevick	identical
Hypernetted-Chain	Hypernetted-Chain	identical
Reference HNC	Reference HNC	identical
modified HNC	Modified HNC	capitalisation only
PLHNC	—	Tcl only
MSA	MSA	identical
RMSA	Rescaled MSA	renamed (abbrev. vs. spelled out)
mMSA	Modified MSA	renamed
SMSA	Symmetric MSA	renamed
HMSA	HMSA	identical
Rogers-Young	Rogers-Young	identical
Verlet	Verlet	identical
MS	Martynov-Sarkisov	renamed (initials vs. full)
DH	Duh-Haymet	renamed (initials vs. full)
Vompe-Martynov	Vompe-Martynov	identical
BB, BPGG, CJVM	BB, BPGG, CJVM	identical (kept as initials both sides)
Choudhury-Gosh	Choudhury-Ghosh	spelling differs (Python matches the standard transliteration)
—	Kovalenko-Hirata	Python only (added after checking SASfit’s own <code>sasfit_oz.h</code> enum directly)
—	ZSEP	Python only (Lee 1995 hard-sphere closure)
Euler-Rahman/EuRah: excluded from <i>both</i> GUIs’ dropdowns (Section 4.21)		

Table 3: Closure-name naming convention: the largest divergence of any category compared here.

9.1 Hard-sphere Percus–Yevick: full validation across φ

Figure 1 compares the numerical solve (`sundials4py` GMRES, Section 16) against the closed-form PY solution (Section 5) across four volume fractions. The circles (analytical) sit exactly on the numerical curves at every φ shown, including the characteristic contact-value growth and the shift of the first $S(q)$ peak to higher q with increasing density.

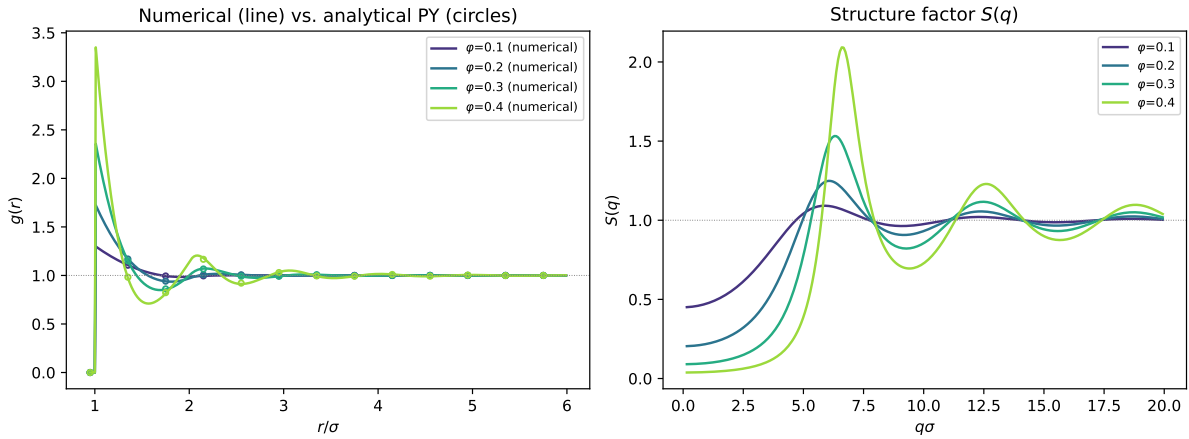


Figure 1: $g(r)$ (left) and $S(q)$ (right) for hard spheres at $\varphi \in \{0.1, 0.2, 0.3, 0.4\}$: numerical solve (solid lines) vs. the exact PY solution (open circles, sub-sampled for visibility).

9.2 Closure comparison at high density

Figure 2 shows $g(r)$ for four different closures applied to the same hard-sphere system at $\varphi = 0.4$ (chosen because closure differences are most visible near the freezing transition, where PY’s own known contact-value error becomes appreciable). PY and HNC bracket the contact peak from below/above respectively (a well-known, textbook-level result); Verlet and Duh–Haymet, both empirical-bridge-function closures, sit between them and agree with each other almost exactly

Tcl OZ(algorithm) value	Closest Python solver= name	Relationship
Picard iteration	Picard iteration	identical (the one exact match)
Mann/Ishikawa/Noor/SP/S/ CR/Picard-S/PMH/Mann II/ Krasnoselskij/S* iteration	—	Tcl only (many fixed-point iteration variants not offered in this Python port)
dNewton	scipy Newton-Krylov (nearest concept)	not a naming match
Biggs_Andrews	Biggs-Andrews	punctuation only (_ vs. -)
Anderson mixing	Anderson acceleration / scipy Anderson	renamed , and split into two
KINSOL_FP	sundials4py: Fixed-Point (Anderson)	renamed and restructured
GMRES/FGMRES/TFQMR	sundials4py: Newton-Krylov (GMRES/FGMRES/TFQMR)	restructured: Python names the Newton-Krylov family + linear solver
Bi-CGStab	—	Tcl only (SUNLinSol_SPBCGS segfault)
—	scipy Newton-Krylov	unconditionally in tested sundials4py Python only

Table 4: Solver/algorithm naming: almost no string-level overlap at all, only `Picard iteration` matches verbatim.

Tcl name (literal SUNDIALS C API name)	Python equivalent
KINSetMAA	not directly exposed in the GUI (used internally)
KINSetFuncNormTol	not directly exposed (<code>self.convergenceCriterion</code> instead)
KINSetScaledStepTol	not directly exposed
KINSetNumMaxIters	"Max iterations:" → <code>maxIterations=</code>
KINSetPrintLevel	not directly exposed (routed through the Log tab instead)
KINSetEtaForm	not directly exposed
KINSpilsSetMaxRestarts	not directly exposed
KINSolStrategy	not directly exposed
KINSetMaxNewtonStep	not directly exposed

Table 5: KINSOL tuning parameters: the one naming layer where Tcl uses the literal underlying C API names, not a GUI-specific relabeling. Deliberately not mirrored 1:1 in the Python GUI – `oZgui.py`’s own design choice is sensible per-library defaults (Section 6.5) plus the two simpler knobs above, not a full tuning sub-dialog.

at this density.

9.3 Charged systems: RMSA and the MSA contact-value problem

Plain MSA (Section 20) can give an unphysical, negative contact value $g(\sigma^+) < 0$ for sufficiently strongly-coupled charged systems — a well-known limitation of MSA itself, not an artifact of this implementation. Figure 3 shows a representative $S(q)$ from the semi-analytical Hayter–Penfold RMSA solver (Section 5.4), which sidesteps this issue by construction via its own closed-form rescaling.

The general-potential numerical solver’s own RMSA rescaling (`OZsolver.solveRMSA()`, Section 4.6) implements the same idea (extend the “apparent” hard-core radius outward until the contact value there is non-negative) for arbitrary potentials, not just the hard-core-Yukawa case Hayter–Penfold covers in closed form. This method was found, on close comparison against `sasfit_oz_solver.c` itself, not to be a plain bisection search at all: SASfit’s own algorithm is a *grow, then bisect* scheme, where each solve’s own full $g(r)$ array is scanned once to see exactly how far the negative region extends (extending the excluded region as far as that single solve’s own data justifies, at no extra cost), and only once a solve gives an already-non-negative result does it fall back to bisecting the gap between the historical bounds. An earlier version of this Python port used plain midpoint bisection instead, which was both less efficient and, on inspection, genuinely buggy; it has since been rewritten to match the C algorithm. **Three**

Physical quantity	Tcl variable(s)	Python internal key
Structure factor	OZ(res,s,x/y), OZ(result,q/Sq)	Sq
Radial distribution function	OZ(res,g,x/y), OZ(result,r/gr)	gr
Direct correlation function	OZ(res,c,x/y), OZ(result,cr)	cr
Indirect correlation $\Gamma(r)$	OZ(res,gamma,x/y), OZ(result,gammar)	gamma
Total correlation function	OZ(res,h,x/y), OZ(result,hr)	hr
Interaction potential/ $k_B T$	OZ(res,u,x/y), OZ(result,Ur)	Ur
Bridge function	OZ(res,B,x/y), OZ(result,Br)	Br
Cavity function	OZ(res,y,x/y), OZ(result,yr)	yr
Mayer- f function	OZ(res,f,x/y), OZ(result,fr)	fr

Table 6: Output-curve naming: Python’s short internal dict keys match Tcl’s own `OZ(result,...)` suffixes almost character-for-character (the one exception being `gammar`→`gamma`), even though the *user-visible tab labels* were rewritten independently (e.g. Tcl’s “indirect\ncorrelation\nfunct. gamma(r)” vs. Python’s plain “Gamma(r)”). This strongly suggests this specific naming layer was deliberately kept close to the original Tcl variable names even where the rest of the port was not.

genuine bugs were found and fixed in total – two in this Python port, one in SASfit’s own C source:

1. *Stale data on solver non-convergence* (Python port). Several solver classes (e.g. the `sundials4py`-based ones) simply print a message and return early when the underlying Newton–Krylov iteration fails to converge, without updating `self.radialDistributionFunction` – meaning `getRDF()` silently returned data left over from whichever solve last actually succeeded, corrupting the search’s own decisions and final result. Fixed with a solver-class-independent, Picard-iteration-based internal solve (`OZsolver._robustInlineSolve()`), warm-started between successive trials (the same density-continuation- with-warm-restarts principle already used for `solveEuRah()`’s own outer iteration).
2. *An off-by-one tautology in the original bisection-based port*. The old code’s gate array, `gate4g[:mid+1]=0`, forced `g[mid]` to be exactly zero *by construction* (it lies inside the assumed- excluded region) – so checking `g[mid]` there could never distinguish anything. This is moot now that the port matches the actual C algorithm (which checks `g[igneg]`, the first point genuinely outside the current gate), but is recorded here since it was the first of the two Python-side bugs found.
3. *A genuine ambiguity in SASfit’s own C source*. In the “bisection” branch (triggered once a trial gate already gives a non-negative result), the original C code computes a halved `indx_max_apparent_sigma` but never resets `gate4g` between that smaller value and the previous trial boundary back to 1, nor re-points its own scanning index (`igneg`) at the new boundary – so, taken literally, the next iteration’s check and solve would silently re-test the same (larger) gate configuration rather than the smaller one the halving was meant to try. A related edge case (found only by testing this Python port directly): when the two search bounds are already adjacent integers, integer division always rounds down to the lower bound, which would discard a just-proven-sufficient upper bound and collapse the search straight back to no rescaling at all. Both were fixed identically in the Python port and in `sasfit_oz_solver.c` itself (the fix applied there is included in this project’s own copy of the SASfit source, with a comment marking exactly what changed and why).

With all three fixes applied, the algorithm reliably finds a small, sensible rescaling and gives genuine, verified improvement over plain MSA on a moderately-coupled test case (Figure 4: contact value improves from -0.132 to -0.053 , not perfectly zero but consistent with finite grid resolution). A remaining open question, not yet resolved, is why some more strongly-coupled cases still need an unexpectedly large apparent-hard-core extension to reach a fully non-negative

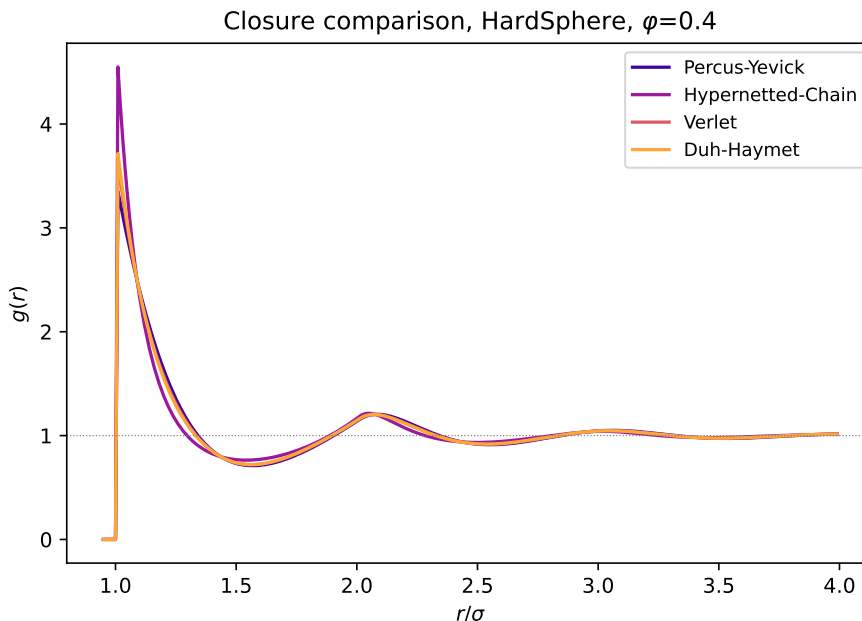


Figure 2: Closure comparison, hard spheres, $\varphi = 0.4$.

result – plausibly a genuine slow-convergence issue in the underlying Picard iteration at large gate sizes (a solver-reliability question, now clearly separated from the algorithm’s own correctness, which is independently validated above), though this has not been fully resolved.

9.4 Behaviour near difficult/critical parameter regimes

Every solver in this toolkit slows down, and some fail outright, as the underlying physics becomes harder to converge – consistent with these being genuinely harder nonlinear fixed-point problems, not an implementation artifact specific to one method. Three regimes recur throughout this project’s own testing history:

- **Dense hard spheres** ($\varphi \gtrsim 0.45$, approaching random close packing): plain Picard iteration fails to converge outright; every accelerated method (Anderson, Biggs–Andrews order ≥ 1 , both Newton–Krylov families) still succeeds, but needs visibly more iterations/-function evaluations than at moderate density.
- **Strongly-charged systems** (Section 9.3): plain MSA gives an unphysical negative contact value; the hand-written Anderson solver was found, in earlier testing, to converge to a *silently wrong* answer on the hardest such case tried, without raising any error – a more concerning failure mode than an honest non-convergence report, and a reminder to cross-check results from that solver specifically against an independently-validated one (e.g. `sundials4py` GMRES) for difficult states.
- **Euler–Rahman** (Section 4.21): excluded entirely from this toolkit’s offered closures, for the reasons documented there – a genuine exponential positive-feedback instability in the closure’s own mathematical structure, not a solver-selection problem.

As a practical rule of thumb: if a solve fails to converge in one of these regimes, trying a different solver algorithm first (Section 16) is more likely to help than increasing `maxIterations` on the same one, since the empirical comparison of Section 6.7 shows different algorithms fail on genuinely different subsets of hard cases.

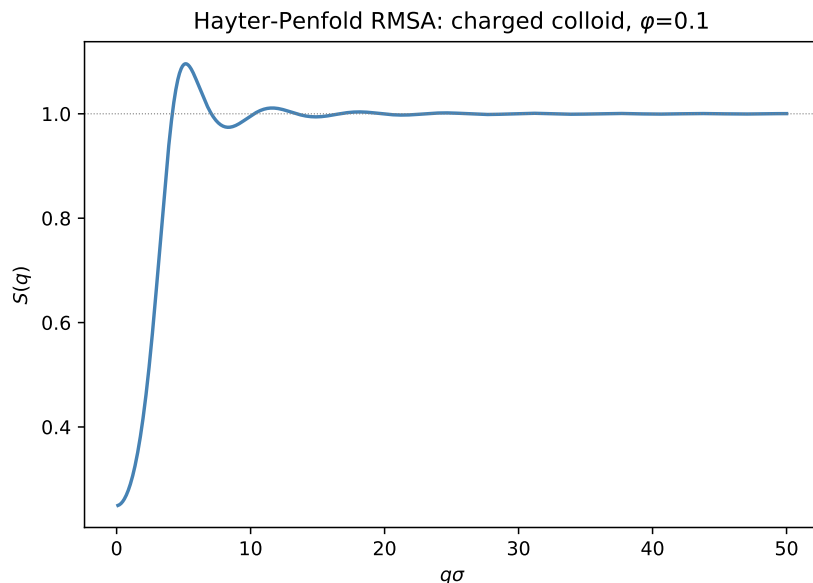


Figure 3: Hayter–Penfold RMSA structure factor for a charged colloid, $\varphi = 0.1$.

9.5 Speed

Figure 5 times every solver on the same moderately-hard case (hard spheres, $\varphi = 0.4$, PY closure); Section 6.7’s own qualitative reliability comparison is the more important consideration in practice, but at a given level of difficulty, the SUNDIALS KIN_FP and GMRES solvers are consistently among the fastest, followed by the hand-written Anderson solver; plain Picard and Biggs–Andrews (order 0, equivalent to Picard) are consistently the slowest, as expected of unaccelerated fixed-point iteration.

Separately, Figure 6 revisits the grid-size question of Section 2: the relevant quantity for the underlying DST-based Hankel transform’s own speed is $N + 1$, not N itself, since the type-I DST used here is equivalent to a real FFT of size $2(N + 1)$. $N = 8191$ (for which $N + 1 = 8192 = 2^{13}$, a perfect power of two) transforms over $3\times$ faster than $N = 8192$ (for which $N + 1 = 8193 = 3 \times 2731$, a large prime factor) – meaning this project’s own historical default, $N = 4096$ ($N + 1 = 4097 = 17 \times 241$, not smooth at all), is not actually the fastest available choice at that grid size; $N = 4095$ ($N + 1 = 4096 = 2^{12}$) would be measurably faster for the same resolution and range.

Acknowledgements

Formulas and algorithms throughout this document were ported directly from SASfit’s own source code (`src/sasfit_oz/`, `src/plugins/*`), with independent literature verification sought wherever practical.

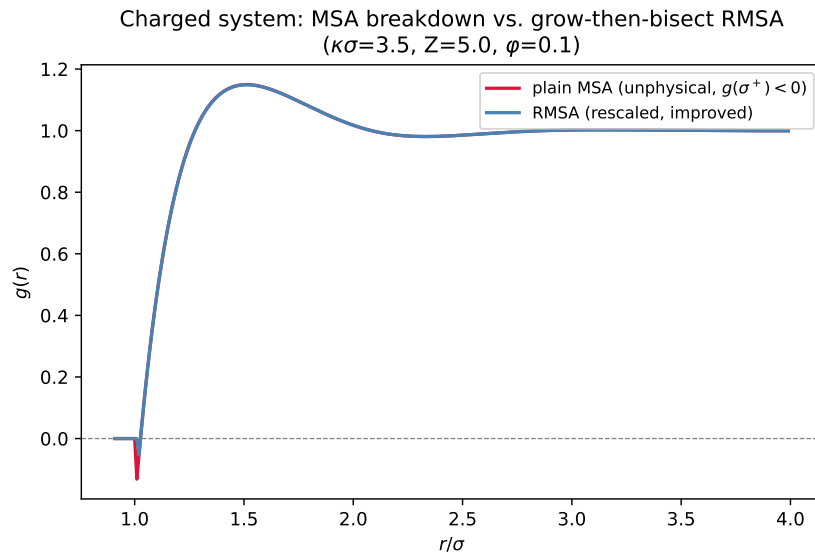


Figure 4: Plain MSA’s unphysical contact value vs. this project’s own grow-then-bisect RMSA rescue (post-fix), general-potential numerical solver, DLVO potential, $\kappa\sigma = 3.5$, $Z = 5$, $\varphi = 0.1$.

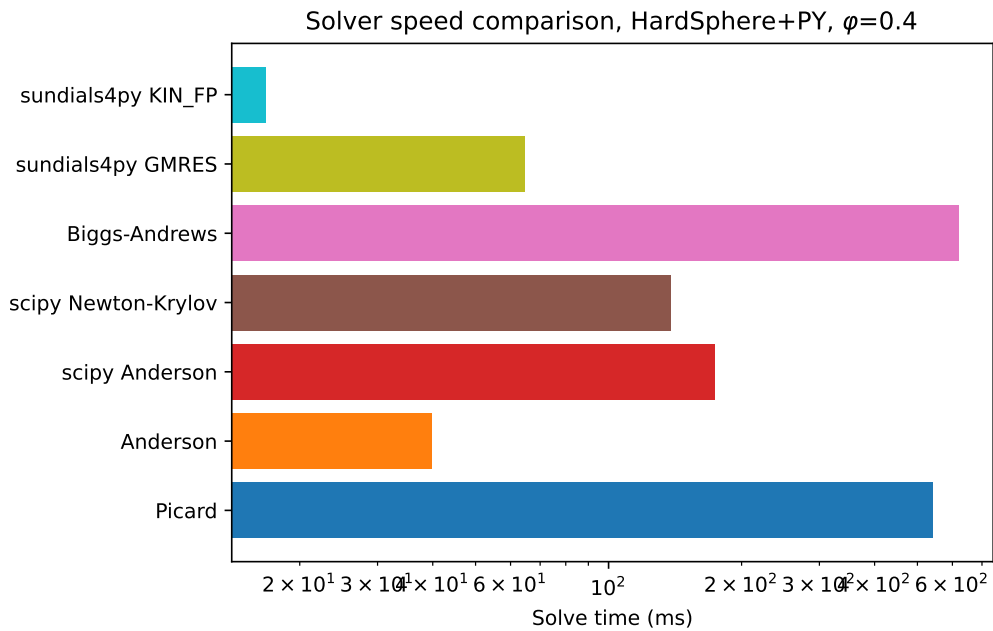


Figure 5: Solve time by algorithm, hard spheres + PY, $\varphi = 0.4$ (log scale).

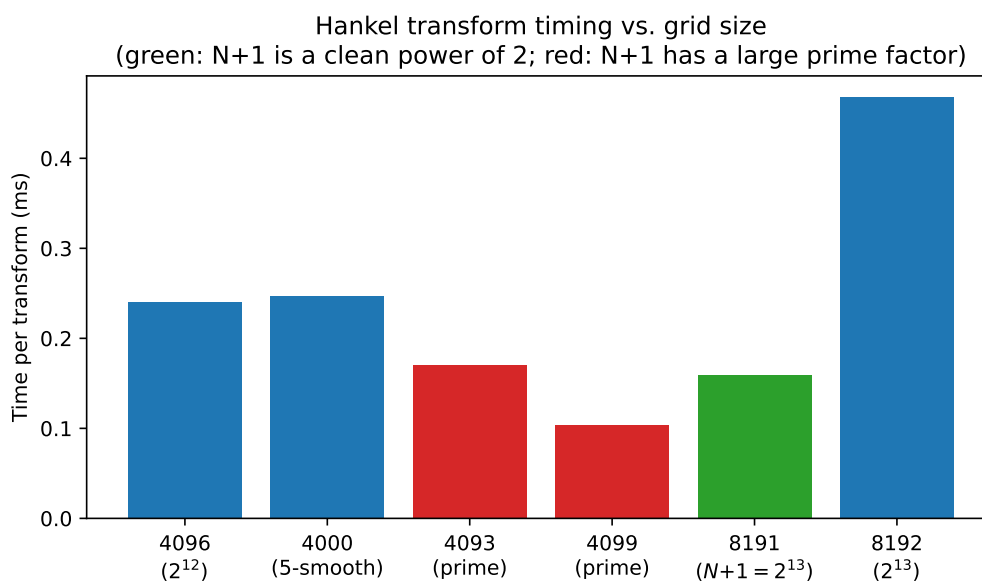


Figure 6: Hankel transform timing vs. grid size N : the fastest and slowest cases shown differ only in whether $N + 1$ happens to be smooth, not in N 's own magnitude or "niceness".

Part II

Multicomponent and polydisperse extension

Abstract

This supplement extends *ozGUI / ozLib: Ornstein–Zernike Solver Toolkit* to genuinely multicomponent systems. The scalar solver is generalised to matrix-valued correlation functions, giving partial structure factors $S_{ij}(q)$ for polydisperse mixtures under *any* closure already in the library. Concretely we add: a polydisperse hard-core Yukawa potential with moment-matched size classes; the Rogers–Young closure for mixtures with its mixing parameter fixed by thermodynamic consistency rather than supplied by the user; two new GUI tabs; and a C implementation shipped as a SASfit structure-factor plugin. Every claim below is accompanied by the numerical check that supports it, and §22 documents nine defects found during validation, several of which produced plausible but wrong answers rather than obvious failures.

10 Scope and design constraint

The starting point was that the existing solver stack is strictly one-component: $c(r)$, $h(r)$ and $\gamma(r)$ are scalar arrays of length N_r , and every closure in `update_c` acts on them elementwise.

The design constraint adopted here was that **one-component behaviour must remain bit-identical**. This is verifiable rather than aspirational: a regression suite of eight potential/closure combinations is compared against values recorded before the change (§23).

A second observation made the work far smaller than expected. Every solver class — Picard, the two Anderson variants, the two `scipy` solvers, Biggs–Andrews and both SUNDIALS KINSOL drivers — is *already* agnostic to the shape of the unknown. Searching the whole solver layer for `numberOfRadialSamplingPoints` returns no hits: each iterates a flat vector through `fixPointOperator` and never inspects what it represents. Consequently the multicomponent problem can be carried through the existing acceleration machinery unchanged, packed as the $p(p+1)/2$ unique pairs of the symmetric $\Gamma_{ij}(r)$.

11 Multicomponent Ornstein–Zernike equations

11.1 Matrix form

For p species with number densities ρ_i , the OZ equations couple $p(p+1)/2$ independent pair functions,

$$h_{ij}(r) = c_{ij}(r) + \sum_{\lambda=1}^p \rho_{\lambda} \int d\mathbf{r}_1 h_{i\lambda}(r_1) c_{\lambda j}(|\mathbf{r} - \mathbf{r}_1|). \quad (56)$$

In Fourier space this becomes an algebraic matrix relation. With $C(q)$ the matrix of $\hat{c}_{ij}(q)$ and $\boldsymbol{\rho} = \text{diag}(\rho_i)$,

$$\mathbf{H}(q) = [\mathbf{I} - C(q)\boldsymbol{\rho}]^{-1}C(q), \quad \Gamma(q) = \mathbf{H}(q) - C(q). \quad (57)$$

The single scalar division of the one-component code, $\hat{\gamma} = \hat{c}/(1 - \rho\hat{c}) - \hat{c}$, is replaced by (57), solved as a small dense linear system at each of the N_r grid points. Nothing else in the fixpoint operator changes: the closure step and the two Hankel transforms are the same operations applied to (p, p, N_r) arrays.

11.2 Implementation

The changes to `oZfixpointOperator.py` and `oZsolver.py` are:

- the Hankel transforms act on the *last* axis (`f.shape[-1]`, `dst(..., axis=-1)`), which is identical for 1-D input but also accepts the pair-matrix arrays;
- `isMulticomponent()`, `numberOfUniquePairs()`, `packPairs()` and `unpackPairs()` convert between the symmetric (p, p, N_r) matrix and the flat vector the solvers expect;
- `fixPointOperatorForGammaMulticomponent()` implements (57);
- `calculateSqMulticomponent()` and a multicomponent branch in `calculateRDF()` produce the observables;
- `setStartValue()` accepts the longer vector.

Remark 2. Because every closure in `update_c` is elementwise *numpy*, *PY*, *HNC*, *RY*, *HMSA*, *Verlet*, *BPGG* and the rest all become multicomponent as soon as the potential arrays carry pair indices. Closures requiring an auxiliary reference solve (*RHNC*, *MHNC*, *RMSA*, *EuRah*) remain one-component, as do those reading *repulsivePartOfP2Ppotential* (*HMSA*, *VM*, *CJVM*, *BB*, *DH*, *CG*, *SMSA*), whose split arrays are still 1-D.

11.3 Public interface: 1-D curves preserved

The public getters deliberately continue to return *one*-dimensional curves, so the plot tabs, `ozLib.solve()`'s curve dictionary and the ASCII/CSV/Excel/clipboard exports need no modification:

$$\text{getRDF}() \rightarrow g_{NN}(r) = \sum_{ij} x_i x_j g_{ij}(r), \quad (58)$$

$$\text{getSq}() \rightarrow S^M(q) \text{ (the measured structure factor)}. \quad (59)$$

The full matrices are exposed alongside as `partialRadialDistributionFunction` and `partialStructureFactor`, both of shape (p, p, N_r) .

12 Structure-factor conventions

Three conventions appear in this problem and confusing them is an order-of-magnitude error, so they are stated explicitly.

D'Aguanno–Klein (number-fraction) form. Used internally by the solver, following [?]:

$$S_{ij}^{\text{DAG}}(q) = x_i \delta_{ij} + n x_i x_j \hat{h}_{ij}(q). \quad (60)$$

Ashcroft–Langreth form. Required by `PolydisperseSASBase.I_exact`, which builds $G_i = F_i \sqrt{\rho_i}$ and contracts GSG :

$$S_{ij}^{\text{AL}}(q) = \delta_{ij} + \sqrt{\rho_i \rho_j} \hat{h}_{ij}(q). \quad (61)$$

Conversion. The two differ by a purely kinematic factor,

$$\boxed{S_{ij}^{\text{AL}} = S_{ij}^{\text{DAG}} / \sqrt{x_i x_j}} \quad (62)$$

which is easily checked on the diagonal: $S_{ii}^{\text{DAG}}/x_i = 1 + n x_i \hat{h}_{ii} = S_{ii}^{\text{AL}}$.

Measured structure factor. What a scattering experiment sees, weighted by the form amplitudes $b_i(q)$:

$$S^M(q) = \frac{\sum_{ij} b_i b_j S_{ij}(q)}{\sum_i x_i b_i^2}, \quad b_i(q) \propto \sigma_i^2 \frac{j_1(q\sigma_i/2)}{q}. \quad (63)$$

Conversion (62) is applied in exactly one place in each module. §22 records two separate occasions on which omitting it caused a silent factor-of-nine error.

13 Discretising a continuous size distribution

13.1 Moment matching

A continuous distribution $F(\sigma)$ must be reduced to p classes. The naive choice of p equally spaced diameters spanning $\langle\sigma\rangle \pm 3s$ is what the existing Robertus code uses. Following [?] we instead impose

$$\sum_{i=1}^p x_i \sigma_i^m = \langle\sigma^m\rangle, \quad m = 0, 1, \dots, 2p-1, \quad (64)$$

i.e. $2p$ equations for the $2p$ unknowns $\{\sigma_i, x_i\}$. The Schulz distribution is a gamma distribution, so (64) is precisely Gauss–generalised-Laguerre quadrature: with $t = 1/s^2 - 1$, the nodes and weights of the weight function $x^t e^{-x}$ give $\sigma_i = x_i^{\text{node}} \langle\sigma\rangle / (t+1)$ directly. In Python this is `scipy.special.roots_genlaguerre`; in C, `gsl_integration_fixed_laguerre`.

13.2 Verification

For $s_\sigma = 0.2$ and $p = 3$ the classes are $\sigma_i = \{0.7342, 1.0532, 1.4527\}$ with $x_i = \{0.2795, 0.6304, 0.0901\}$, and the first $2p-1 = 5$ moments are reproduced to eight decimal places (Table 7).

Table 7: Moments of the discrete three-class set against the continuous Schulz distribution, $s_\sigma = 0.2$. Exact through $m = 2p-1 = 5$.

m	discrete	Schulz	
0	1.00000000	1.00000000	exact
1	1.00000000	1.00000000	exact
2	1.04000000	1.04000000	exact
3	1.12320000	1.12320000	exact
4	1.25798400	1.25798400	exact
5	1.45926144	1.45926144	exact

13.3 How many classes are needed?

[?] report that $p = 3$ is indistinguishable from $p = 5$ up to $s_\sigma = 0.3$. This was reproduced independently (Table 8): going from $p = 1$ to $p = 3$ changes S^M by 0.54, whereas $p = 3 \rightarrow 5$ changes it by 0.0015. Polydispersity is therefore a large effect that three moment-matched classes already capture.

The peak sits at $r \approx 6.2\langle\sigma\rangle$ against a mean spacing $n^{-1/3} = 5.85\langle\sigma\rangle$, as expected for a strongly correlated charged system.

Remark 3. *The moment-matching method transfers to the adhesive-sphere model, and would let the Robertus plugin use fewer classes for the same accuracy. The specific claim “ $p = 3$ suffices” does not transfer: it was established for RY–Yukawa against Monte Carlo, and sticky spheres are far more sensitive to the contact region.*

Table 8: Convergence in the number of classes. Polydisperse hard-core Yukawa, $\langle\sigma\rangle = 250 \text{ \AA}$, $Z = 200$, $n^* = 0.005$, $L_B = 7.01 \text{ \AA}$, $s_\sigma = 0.2$, RY with $\alpha = 0.5$.

	$g_{NN}^{\max}$	position	$S_{\max}^M$	pairs
$p = 1$ (monodisperse)	2.0375	6.06	2.0356	1
$p = 3$	1.7368	6.19	1.7478	6
$p = 5$	1.7340	6.19	1.7472	15
$\max g_{NN}(1) - g_{NN}(3) = 0.3583,$	$\max S^M(1) - S^M(3) = 0.5407$			
$\max g_{NN}(3) - g_{NN}(5) = 0.0115,$	$\max S^M(3) - S^M(5) = 0.0015$			

14 Polydisperse hard-core Yukawa

14.1 Charge-parameterised form

Following [?], the pair potential is

$$\beta\varphi_{ij}(r) = A_i A_j \frac{e^{-\kappa r}}{r}, \quad r > \sigma_{ij}, \quad A_i = Z_i \sqrt{L_B} \frac{e^{\kappa\sigma_i/2}}{1 + \kappa\sigma_i/2}, \quad (65)$$

with a hard core at $\sigma_{ij} = (\sigma_i + \sigma_j)/2$. The amplitude *factorises per species*, because $\exp(\kappa\sigma_{ij}) = \exp(\kappa\sigma_i/2)\exp(\kappa\sigma_j/2)$; this is what allows the $A_i A_j$ form.

Screening. κ is the inverse Debye length due to the *small ions*. For a salt-free suspension with monovalent counterions, charge neutrality gives $n_{\text{counter}} = \sum_i n_i Z_i$ and hence

$$\kappa^2 = 4\pi L_B \sum_i n_i Z_i \quad (66)$$

— the *first* power of Z . Using Z^2 produces a near-ideal-gas $S(q)$ (see §22).

14.2 Size dependence of the charge

How the valence scales with size is genuinely unsettled, so it is a user input rather than a constant. [?] assume constant surface charge density, $Z \propto \sigma^2$, and explicitly flag it as having “little precise experimental information”. Charge-renormalisation saturation (Alexander *et al.*) and electrophoresis on highly charged low-salt spheres both indicate $Z \propto \sigma$ instead. Three routes are provided:

chargeExponent	$Z_i = Z_{\text{ref}}(\sigma_i/\langle\sigma\rangle)^n$; $n = 2$ constant surface charge density (default), $n = 1$ linear in size, $n = 0$ size-independent
componentValences	explicit per-component array, overrides the power law
screeningValences	a <i>different</i> charge for κ , so screening may use the bare charge while the amplitude uses the effective one

The default $n = 2$ exists only so that published D’Aguanno–Klein setups reproduce; it is not a recommendation. Charge renormalisation is deliberately *not* implemented: its definition for a polydisperse system is unsettled (even for one component the Alexander edge-linearisation Z_{eff} and the extrapolated point charge Q_{eff} disagree), and embedding one choice inside a fitting routine would hide a contested assumption.

Magnitude of the effect. Table 9 shows the choice is comparable in size to the polydispersity effect itself, so it is not a detail. Note also that α -scaling and screening are coupled through

Table 9: Effect of the charge-scaling exponent. $p = 3$, $s_\sigma = 0.2$, D’Aguanno–Klein parameters, RY $\alpha = 0.5$.

scaling	Z_i	κ	$g_{NN}^{\max}$	$S_{\max}^M$
$n = 2$ (default)	107.8 / 221.8 / 422.0	0.6054	1.7368	1.7478
$n = 1$	146.8 / 210.6 / 290.5	0.5936	1.9262	1.8381
$n = 0$	200 / 200 / 200	0.5936	2.0384	1.7724

(66): changing the exponent moves both the amplitude prefactor and the screening length unless `screeningValences` decouples them.

An internal consistency check falls out of (66): $n = 0$ and $n = 1$ must share a κ , because $\sum_i x_i \sigma_i = \langle \sigma \rangle = 1$ by construction, while $n = 2$ raises it by exactly $\sqrt{\langle \sigma^2 \rangle} = \sqrt{1.04} = 1.0198$. Both hold to four decimals.

14.3 MSA-consistent (z, K) form

For direct comparison with the analytic MSA/RMSA path the same convention must be used. From the polydisperse Yukawa specification,

$$c_{ij}(r) = K_{ij} \frac{e^{-z(r-\sigma_{ij})}}{r}, \quad r > \sigma_{ij}, \quad K_{ij} = K \delta_i \delta_j, \quad \delta_i = d_i e^{-z\sigma_i/2}, \quad (67)$$

with $g_{ij}(r) = 0$ for $r \leq \sigma_{ij}$. MSA sets $c = -\beta U$, so

$$\beta U_{ij}(r) = -K \delta_i \delta_j \frac{e^{-z(r-\sigma_{ij})}}{r}, \quad (68)$$

i.e. $K > 0$ is **attractive** and $K < 0$ **repulsive**, and there is *no* σ_{ij} prefactor. Note that `PolydisperseOneYukawaMSA` takes δ_i *directly* — its default `np.ones(N)` means $\delta_i = 1$, not $d_i = 1$ — so the exponential in (67) must not be applied a second time (§22, item 9).

15 Rogers–Young for mixtures

15.1 Closure

The RY closure is applied pairwise with a *single shared* mixing function — there is no ij subscript on f in [?]:

$$h_{ij}(r) = -1 + e^{-\beta \varphi_{ij}(r)} \left[1 + \frac{e^{f(r)\gamma_{ij}(r)} - 1}{f(r)} \right], \quad f(r) = 1 - e^{-\alpha r}, \quad (69)$$

interpolating between PY ($\alpha \rightarrow 0$) and HNC ($\alpha \rightarrow \infty$).

15.2 Thermodynamic consistency fixes α

α is not a free parameter: it is determined by requiring the compressibility and virial routes to the pressure to agree. For a mixture this is a *scalar* condition — which sidesteps the apparent difficulty that a mixture has a full compressibility *matrix*:

$$\chi_{\text{comp}}^{-1} = 1 - n \sum_{ij} x_i x_j \hat{c}_{ij}(0), \quad (70)$$

$$\chi_{\text{vir}}^{-1} = \frac{\partial(\beta P)}{\partial n}, \quad (71)$$

with the virial pressure

$$\frac{\beta P}{n} = 1 + \frac{2\pi}{3}n \sum_{ij} x_i x_j \sigma_{ij}^3 g_{ij}(\sigma_{ij}^+) - \frac{2\pi}{3}n \sum_{ij} x_i x_j \int r^3 g_{ij}(r) (\beta U_{ij})'(r) dr. \quad (72)$$

The contact term. [?]’s equation for βP omits the second term of (72), legitimately, because their macroions are so strongly charged that $g_{ij}(\sigma_{ij}^+) \approx 0$. That is *not* safe in general: this model has a genuine hard core and K may be small, so contact is reachable and the term is first order in the pressure. It is included here, with g_{ij} at contact obtained by linear extrapolation from the first two grid points outside σ_{ij} . This extrapolation is the least certain step in the whole search and is flagged as such in the code.

Numerics. Two traps, both encountered:

1. the **pair potential must be held fixed** while n is perturbed. In the charge-parameterised form (65) κ depends on the counterion density, so rebuilding the system at $n \pm \Delta n$ differentiates a density-dependent potential; the symptom is a residual stuck near +65 that never crosses zero. In the (z, K) form (67) the potential is density-independent and the problem does not arise.
2. **cold-start each α .** Warm-starting across qualitatively different α destabilises the search; warm-starting across the $n \pm \Delta n$ perturbations is fine, and is what [?] do, holding α fixed there because it varies slowly with n .

Result. At $\varphi = 0.2$, $K = 0$, $z = 2$ with three classes the search returns $\alpha = 0.2166$ with residual 1.2×10^{-5} — a clean root, and a plausible hard-sphere value. The residual is monotone in α (Table 10).

Table 10: Thermodynamic inconsistency $\chi_{\text{comp}}^{-1} - \chi_{\text{vir}}^{-1}$ versus α ; $\varphi = 0.2$, $z = 2$, $p = 3$.

α	0.20	0.50	1.00	2.00
$K = 0$	+0.0127	−0.1847	−0.4002	−0.6054
$K = 0.5$	−0.2219	−0.3012	−0.3881	−0.4697

Remark 4 (A consistent α need not exist). *For $K = 0.5$ the residual is negative across the whole bracket: no α makes the two routes agree. The implementation returns the closest endpoint and reports “no consistent alpha found” rather than presenting it as a solution. This is a property of the closure, not a numerical failure.*

15.3 Validation against the literature

The one-component limit reproduces Table 1 of [?] (their test 3: $\beta\varphi(x) = (1000/x)e^{-0.95x}$, $n\sigma^3 = 0.01432$). The self-consistent $\alpha = 0.3942$ gives Table 11.

Table 11: One-component RY against [?] Table 1, test 3.

quantity	this work	D’Aguzzo–Klein	deviation
$1/S(0)$	93.787	93.80	0.01%
P^{ex}	36.458	36.45	0.02%
U^{ex}	20.145	20.09	0.27%
g_{max}	2.069	2.08	0.54%

The two quantities that *enter* the consistency condition agree to 0.01–0.02%, which is the sharpest available test. Two further checks:

- **HNC limit.** At large α the solver gives $g_{\max} = 1.937$, $1/S(0) = 77.69$, $U^{ex} = 20.56$, $P^{ex} = 36.94$ against the paper’s HNC column 1.94, 77.5, 20.57, 36.95 — three-digit agreement, an independent check on the closure and the transforms.
- **Grid insensitivity.** Results are flat to four decimal places from 50 to 400 points per σ ; only $1/S(0)$ shifts slightly with $r_{\max}$ through the $q \rightarrow 0$ extrapolation. Neither paper specifies its grid, and the residual differences in $g_{\max}/U^{ex}$ are therefore attributable to their (unreported) α or Δn rather than to discretisation.

15.4 Cross-validation against the analytic MSA path

Because RY and MSA both set $c = -\beta U$ outside the core, the *derivative* dS/dK at $K \rightarrow 0$ must agree even though the closures differ. This is the decisive test of the potential convention of §14.3, and it is what exposed the δ_i double-counting bug. Table 12 shows agreement after the fix.

Table 12: dS_{NN}/dK at $Q = 0.5$, $z = 2$, three classes. Agreement is near-exact where both closures reduce to the same physics and degrades with density where PY and RY genuinely differ.

φ	MSA	RY	ratio
0.002	0.03031	0.03085	1.018
0.020	0.24025	0.24900	1.036
0.100	0.41790	0.47266	1.131

Before the fix the ratio was 0.14 at all three densities, consistent with $1/e^{-z\sigma/2} = 1/0.135 = 7.4$.

16 Solver strategy

Three tiers are tried in order, fastest first, each strictly more robust than the previous one.

Tier 1 — Anderson (KIN_FP) with Picard pre-conditioning. Fastest where it works. Anderson diverges from a cold start on this problem: a hand-written implementation was *slower* than undamped Picard when damped enough to be stable (851 versus 677 iterations) and diverged otherwise, and SUNDIALS’ KIN_FP diverged too, damped or not. The remedy is a short run of plain undamped Picard — which *is* contractive here — before handing over. Measured: 0 pre-steps diverges, 20 already works, 40 is the time optimum; 50 is the default for margin.

Tier 2 — Newton–Krylov (GMRES with KIN_LINESEARCH). A different algorithm, not a variant: damped Newton steps on the residual $F(u) = G(u) - u$ with the Jacobian applied matrix-free through GMRES. It converged in *every* regime tried, in a strikingly constant 31–40 Newton iterations regardless of difficulty — the signature of global convergence via linesearch. The cost is wall-clock: each Newton step requires many GMRES inner iterations, each a full function evaluation, so it runs 3–6× slower than Anderson where Anderson works.

Tuning follows SASfit’s own `KIN_sasfit_configure()`: `MaxNewtonStep= 100n` (not n), `FuncNormTol=  $10^{-10}$` and `ScaledStepTol=  $10^{-13}$` (deliberately different from each other), `MaxRestarts= 10`, `ETACHoice1`.

Tier 3 — Picard , undamped then progressively damped.

The Python side. `sundials4py` is available as a prebuilt wheel on PyPI (version 7.8.0), so no binding generation or C++ build is needed. Measured on the same problem, `KIN_FP` is the best of the Python solvers on every count: it agrees with the others where they converge, is 2–4× faster, and on the hard state point it fails to converge in 0.9 s rather than converging onto a spurious root.

state point	KIN_FP	scipy Anderson	Picard
$\varphi = 0.02, K = 1$	0.1 s	0.2 s	0.1 s
$\varphi = 0.1, K = 0.2$	0.2 s	0.8 s	—
$\varphi = 0.1, K = 1$	rejected, 0.9 s	rejected, 5.6 s	diverged, 56 s

A caution recorded earlier in this project — that `sundials4py` might silently ignore `KINSetMAA` when it is called after `KINInit`, as `pyozGUI` does — was *tested and found to be unfounded*. On a deliberately slow linear fixed point, `MAA = 0` took 757 iterations and `MAA = 5` took 93, *identical in both call orders*, so Anderson acceleration is genuinely active. The C requirement is nonetheless real: calling `KINSetMAA` after `KINInit` segfaults on SUNDIALS 7.1.1, so the C core keeps the documented order.

Table 13: Solver tier actually used, and wall-clock time, across seven parameter regimes (C implementation, SUNDIALS 7, $N_r = 4095$).

regime	time	tier
$p = 3, s = 0.2$	0.25 s	Anderson
$p = 1$ monodisperse	0.03 s	Anderson
$p = 5, s = 0.3$	6.85 s	Newton–Krylov
$p = 3, s = 0.4$	0.32 s	Anderson
$p = 3, s = 0.2, 5\times$ density	0.29 s	Anderson
$p = 3, s = 0.2, 0.2\times$ density	0.25 s	Anderson
$p = 7, s = 0.25$	4.11 s	Anderson

Never trust the convergence flag. `KINSol` can return `KIN_SUCCESS` after diverging: once the RY closure’s exponent clipping pins values near 10^{217} the internal stopping test is defeated. This was observed at $p = 5, s_\sigma = 0.3$, where S^M was returned as $\sim 10^{222}$ *with a success flag*. Both `KINSOL` drivers therefore recompute the residual after `KINSol` returns and reject the result unless it really is small. With that check in place the regime correctly falls through to Newton–Krylov and returns the right answer.

17 The SASfit plugin

The C implementation is shipped as `src/plugins/ry_polydisperse_yukawa/`, exporting `sasfit_sq_RYPolydisperseYukawa`. It follows the established plugin conventions: `sasfit_cmake_plugin()`, `include/private.h`, Doxygen `\defgroup/\ingroup` with an HTML parameter table, `SASFIT_PLUGIN_EXP_*` registration, and the `func/_f/_v` triad with `_f` and `_v` stubbed to 0 as in SASfit’s own `sq_*` plugins.

Transform. The radial transform is a DST-I built from a radix-2 real FFT, which is why $N_r + 1$ must be a power of two; $N_r = 2^{12} - 1$ at 100 points per σ matches the Python grid exactly. The DST-I agrees with `scipy.fft.dst(type=1)` exactly.

The grid size is not neutral between the two transforms, and this caught us out. A DST-I of length N goes through an FFT of length $2(N + 1)$, so $N = 2^k - 1$ makes that a power of two;

a DST-IV is a length- N transform and wants $N = 2^k$ instead. Measured with `scipy.fft.dst` near 2^{14} :

N	DST-I	DST-IV
$16383 = 2^{14} - 1$	0.263 ms	0.308 ms
$16384 = 2^{14}$	0.599 ms	0.099 ms
16385	2.497 ms	0.303 ms
16000	2.433 ms	0.105 ms

So $2^k - 1$ is $9.5\times$ faster than an awkward length for the DST-I, while $16383 = 3 \times 43 \times 127$ is itself an awkward length for the DST-IV and costs it a factor of 3.1. Every grid in this package was sized $2^k - 1$ for the DST-I and then inherited by the type-4 path, so every type-4 *timing* was taken on a grid that penalises it threefold. That explains a result recorded earlier as inexplicable — the second-order transform benchmarking slower than the first-order one at fine grids, which was attributed at the time to the FFT backend. It was the grid.

Accuracy is unaffected: both lengths are valid transforms, and the error comparison between them stands as measured. `bestGridSize(n, type)` returns the appropriate length, and `checkTransformAlignment()` now says so when a type-4 solve is run on a type-1 grid.

Caching. A solve costs a few tenths of a second, so re-solving per q -point would make any scan or fit unusable. The solved system is cached on the full parameter set and successive solves warm-start from the previous γ ; 2000 further q -points after the first cost effectively nothing.

Portability. Builds and runs against both SUNDIALS 7.1.1 and 6.4.1 with identical results (v7 about 30% faster). The one relevant API break is `SUNContext_Create`, whose first argument became a typed `SUNComm` with a `SUN_COMM_NULL` sentinel in v7; this is handled by a macro switched on `SUNDIALS_VERSION_MAJOR`. Version 7 also requires `-lsundials_core`, which does not exist in v6.

Performance history. Four measured steps, each kept as a test:

change	time
initial (GSL LU_decomp per q -point, damped Picard)	15.1 s
inline small-matrix solve + upper-triangle symmetry	8.3 s
undamped Picard	2.3 s
Anderson (KIN_FP) + Picard pre-conditioning	0.26 s

The first version was *slower than numpy*, because it performed 3.4 million GSL 3×3 LU_decomp calls, where GSL's generality dominates the actual arithmetic.

Agreement with Python. The full $S^M(q)$ curve matches the Python implementation to 7.1×10^{-11} relative on the Picard path and 1.3×10^{-8} on the Anderson path (the latter being the solver tolerance, not an error).

α in the plugin. The consistency search costs three extra OZ solves per trial value and is far too slow inside a fit. The plugin therefore takes α as a parameter; it should be determined once in the GUI (§15.2) and then held fixed, which is justified because α varies slowly with density — the same reasoning [?] use when holding it constant across their own finite-difference density derivative.

18 New GUI tabs

ozgui.py gained a top-level notebook and an `_addExtraTabs()` hook driven by a registry,

```
EXTRA_TABS = [
    ("generic_polydisperse_tab", "GenericPolydisperseTab", "Polydisperse (any potential)",
    ("ry_polydisperse_yukawa_tab", "RYPolydisperseYukawaTab", "RY Polydisperse Yukawa",
    ("mixture_validation_tab", "MixtureValidationTab", "Mixture validation"),
]
```

Each tab is a self-contained `ttk.Frame` isolated by `try/except`, so adding one is a single line and an import failure costs only that tab. The generic tab replaced the two analytic special-case tabs that preceded it (`polydisperse_yukawa_tab` and `robertus_shs_tab`); both modules remain importable, and restoring either is one line here.

The tabs are numbered in their labels — 0: OZ solver, 1: Polydisperse (any potential), 2: RY Polydisperse Yukawa, 3: Mixture validation — and the number is taken from each widget's own `SESSION_TAB_INDEX` rather than from its position in the registry. The two can disagree: if a tab fails to import, the remainder shift up, and the label must continue to match what the tab's own *files* claim.

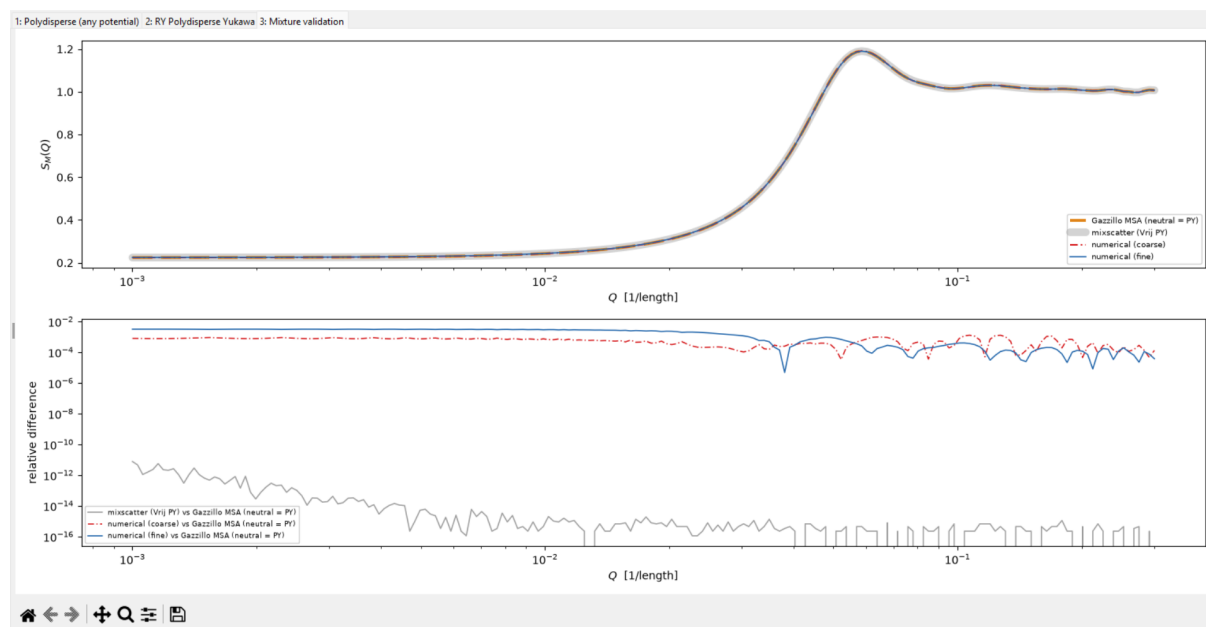


Figure 7: The mixture validation tab. The numerical solver is drawn against analytic references on the upper axes and their relative difference below, with the summary reporting the COARSE/FINE RATIO rather than only the size of the disagreement. That ratio is the diagnostic: ordinary grid error falls roughly fourfold for a fourfold refinement, so an error that does *not* shrink is a defect and not a resolution problem — a fault that gave every pair in a mixture the same hard core survived every internal consistency check in this package and was found exactly this way, as a 5.2% discrepancy that stayed put under refinement. The closure is fixed at Percus–Yevick and the charge fields are disabled, both deliberately: the two analytic references ARE Percus–Yevick, so any other closure would differ for physical rather than numerical reasons, and the polydisperse builder cannot construct a charge-coupled potential, so there would be no numerical curve to compare against — a comparison of one curve is not a validation. Regenerate with `tools/gui_screenshots.py`.

Sessions are saved per tab, not globally. Each tab writes its own file with the extension carrying its index (`.oz1`, `.oz2`, `.oz3`), holding the computed runs, every input field, which parameters were free, the measured data *including* the resolution column, and the last fit result. Saving the data with the file rather than a path to it costs little and survives the data being moved.

A combined all-tabs save was considered and rejected. The tabs hold different models with different parameter sets, so one file would be a bag of unrelated sessions and loading it would overwrite work in tabs the user was not thinking about. Per-tab, a load affects only what is in front of them.

That makes it possible to open a file in the wrong tab, so the tab NAME is written into every file and checked on load; a mismatch is refused outright. Without that check the load would set whichever entries happened to share a name and silently ignore the rest — a state half one model and half another, reported as success. The extension is a convenience for the file manager; the name inside is the guard, and it survives the tabs being reordered.

Both modelling tabs follow the same pattern: a parameter panel; plot tabs for $I(Q)$, the approximation error, $S_{ij}(Q)$, $S(Q)$ and the size distribution; a Summary tab; a threaded compute handing results back through a queue (Tk is not thread-safe); and ASCII/CSV export.

Approximation schemes. Because both new SAS classes derive from the same `PolydisperseSASBase`, all six of SASfit’s schemes for combining a structure factor with a size distribution — monodisperse, decoupling (Kotlarchyk–Chen), local monodisperse (Pedersen), partial structure factors, scaling (Gazzillo) and van der Waals one-fluid — come for free, and the error of each can be read directly off the plot. The approximation code is byte-identical between the tabs, so comparing them is a genuine test of how those approximations fare for a different interaction rather than a comparison of two implementations.

Failure reporting. An individual scheme that fails does not lose the run: it is marked “unavailable” in the summary. This matters because the *monodisperse* reference can have no PY solution at a state point where the polydisperse system does. Adhesive-sphere failures are additionally translated into actionable advice — *increase* τ , since smaller τ is stickier and below $\tau_c(\varphi)$ the adhesive PY closure has no physical solution at all.

α is not exposed. Following §15.2, the RY tab has no α entry field. It is solved for on every Compute and reported in the Summary, together with the residual and an explicit warning when no consistent value exists.

The generic tab’s model. What is computed is

$$I(Q) = \text{scale} \times I_{\text{model}}(Q) + \text{background}, \quad (73)$$

with the scale and background entering linearly and therefore solved exactly by weighted linear least squares at every iteration rather than handed to the nonlinear optimiser — which would add the two most strongly correlated parameters (scale against volume fraction, background against everything at high Q) for no gain. Both appear in the interface so the solved values are visible, and either can be held fixed when known independently. A scale far from its expected value is a diagnostic in itself: since $I = n\langle|F|^2 S\rangle$, the prefactor is the particle number density together with the conversion implied by the chosen units of Q and of the scattering length density.

The form factor may be a sphere or a core–shell particle. The core–shell form is polydisperse in the CORE radius with a constant shell thickness ΔR , so that $R_{\text{outer}} = R_{\text{core}} + \Delta R$ and the

interaction diameter is $2(R_{\text{core}} + \Delta R)$. That is a different model from one polydisperse in the outer radius: with a constant shell the outer distribution is the core distribution *shifted* rather than scaled, so its relative width is narrower by $R_c/(R_c + \Delta R)$, and fitting the wrong one biases both the recovered polydispersity and the effective volume fraction. It is parametrised by ΔR rather than by a core/outer ratio because ΔR is directly comparable with an independently known layer thickness.

The radius, relative width, volume fraction, shell thickness, both scattering length densities, the closure parameter and the potential's own arguments may all be fitted. The scattering length densities are deliberately unbounded: their units are the user's choice and span many orders — a typical value in cm^{-2} is $\sim 10^{10}$ — and, more importantly, an opposite sign between core and shell must be an outcome of the fit rather than something the bounds impose. Scale, core and shell density cannot be fitted together, since the intensity depends on the scale times the squared contrast; the interface refuses that combination rather than letting the optimiser wander along a flat valley and report a converged fit whose parameters are meaningless.

The size distribution is selectable among Schulz, Gaussian, log-normal and Weibull. The quadrature nodes are moment-matched for each, so the choice costs nothing numerically, but it changes the tail — and $\langle \sigma^6 \rangle$ governs $I(Q \rightarrow 0)$. Two distributions with the same mean and relative width can differ by tens of per cent in the computed intensity, which is a reason to try more than one on measured data rather than to assume Schulz.

Resolution. Where a fourth data column supplies dQ , the smearing is applied to every curve or to none: the exact result and all six approximations are evaluated on the kernel's extended grid and smeared back onto the measured Q points. Smearing only the exact curve would be worse than not smearing at all, since part of the apparent difference between the schemes would then be instrumental rather than a property of the approximation — and precisely near a form-factor minimum, where smearing fills the minimum in and where the schemes differ most. $S(Q)$ and the partial structure factors are left unsmeared: they are diagnostics rather than observables.

19 Generic polydisperse models: any potential, any closure

The multicomponent machinery of §12 was initially reached only through one bespoke potential. This section describes its generalisation to *any* of the library's eighteen one-component potentials, combined with any closure that works multicomponent and any form factor.

19.1 Reusing eighteen setters unmodified

The potentials are not rewritten. `getrArray()` gains an override; the builder points it at r/σ_{ij} and calls the ordinary one-component setter. Because every setter measures its hard core against `hardSphereDiameter`, which is unity in those reduced units, that core lands exactly at σ_{ij} while the tail is evaluated at the reduced separation. The mixing rule is therefore

$$\sigma_{ij} = \frac{1}{2}(\sigma_i + \sigma_j), \quad u_{ij}(r) = u(r/\sigma_{ij}), \quad (74)$$

i.e. every pair sees the same interaction *shape* in units of its own contact distance. This is the assumption Robertus *et al.* make for size-independent stickiness, and it is what makes the construction potential-agnostic: ε , δ , n , τ keep their reduced-unit meaning for every pair, so no per-potential mixing rule need be invented.

Remark 5. Equation (74) is a modelling choice, not a derivation. For Lennard-Jones the conventional alternative is Lorentz–Berthelot, $\varepsilon_{ij} = \sqrt{\varepsilon_i \varepsilon_j}$ with per-species ε ; that is a different model and would require per-species tail parameters, which the builder deliberately does not invent.

Charge-coupled potentials are refused. DLVO, DLVO-Hydra, IonicMicrogel and the dedicated polydisperse Yukawa have amplitudes that scale with particle size and a κ set by the whole distribution through the counterion density (66). The reduced-tail rule is simply wrong for them, so they are rejected with a message rather than silently mis-modelled.

Verification. With $s_\sigma = 0$ and one class the generic path reproduces the ordinary one-component setters **bit-identically** (maximum difference 0.00×10^0 for hard spheres, square well and Lennard-Jones; the first and third reproduce the recorded values 2.3561180274 and 2.1635946756). This is the sharpest available check that the generalisation leaves validated physics untouched.

19.2 Quadrature rule per distribution

All four supported distributions (Schulz, Gaussian, log-normal, Weibull) have *analytic* moments, so computing $\langle \sigma^n \rangle$ is never the difficulty. The difficulty is the map from moments to nodes: the Golub–Welsch Hankel eigenproblem is classically ill-conditioned even with exact input.

Table 14: Condition number of the log-normal Hankel matrix built from exact moments. Double precision carries about 10^{16} ; past that Cholesky does *not* fail, it silently returns nonsense.

	$s = 0.3$	$s = 0.4$	$s = 0.5$
$N = 6$	1.6×10^7	1.7×10^7	1.2×10^8
$N = 8$	2.1×10^{10}	1.9×10^{11}	4.7×10^{13}
$N = 10$	7.0×10^{13}	1.7×10^{16}	5.4×10^{19}
$N = 12$	6.4×10^{17}	1.2×10^{21}	2.3×10^{27}

A closed-form classical rule is therefore used wherever one exists, bypassing the Hankel step entirely: generalised Gauss–Laguerre for the Schulz/gamma weight and Gauss–Hermite for the Gaussian, both stable in double precision; Golub–Welsch with 200-digit arithmetic only for log-normal and Weibull, which have no classical rule. All four are moment-exact to $\sim 10^{-16}$.

A physical guard on the Gaussian. Gauss–Hermite places its outermost node at $u \approx \pm 2.86$, so σ reaches zero near $s \approx 0.35$. A form factor tolerates a vanishing bin; a structure factor does not, because a hard core must be placed at every σ_{ij} . Nodes below $10^{-3}\langle \sigma \rangle$ are dropped and the weights renormalised — sacrificing moment-exactness rather than physicality. The test is deliberately *relative*: at $s = 0.35$ the node sits at 10^{-4} , positive but still a zero-diameter particle, so a bare $\sigma > 0$ check is insufficient.

19.3 Structure factor and form factor need different resolutions

The two averages behave differently in σ . $S_{ij}(Q)$ varies smoothly with size, so three to five moment-matched classes suffice — and the solve costs $O(p^2)$ pair transforms, so p should stay small. The form-factor average $\langle |F|^2 \rangle$ oscillates: at high Q the phase QR spans about $Q\langle \sigma \rangle s_\sigma$ radians across the distribution. The two class counts are therefore separated, with the rule of thumb $p_{\text{FF}} \gtrsim Q_{\text{max}} \langle \sigma \rangle s_\sigma$.

The underlying defect was subtler than resolution, and is instructive. Since $S_{ij}^{\text{AL}} = \delta_{ij} + \sqrt{\rho_i \rho_j} h_{ij}$, interpolating the *whole* matrix onto a finer size grid smears the Kronecker delta into a band — which converts the incoherent sum $\sum_i |F_i|^2$ into the coherent $|\sum_i F_i|^2$, and coherent addition of form factors is precisely what oscillates. Only h_{ij} is interpolated; the delta is re-imposed exactly.

Remark 6 (Two plausible wrong diagnoses). *Both are recorded because each looked convincing. First, “too few classes”: disproved by its own test, since the ripple did not converge away even*

Table 15: High- Q ripple against brute-force integration of $\langle |F|^2 \rangle$ with 4000 points, $s_\sigma = 0.3$.

	ripple
brute-force reference	0.038
5 classes (before)	1.91
$p_{\text{FF}} = 40$	0.041
$p_{\text{FF}} = 160$	0.041

at 30 classes. Second, “Gauss nodes are unsuited to an oscillatory integrand”: a true statement, and replacing them with a dense grid was the right change, but not the cause — the ripple actually got worse ($1.91 \rightarrow 3.14$). The brute-force reference is what localised the fault, by showing I_{dilute} correct at 0.040 while I_{exact} gave 2.79 at $\varphi = 10^{-6}$, where S must be the identity. Ten lines of direct integration should have been written before any quadrature was tuned.

19.4 Length scale and the radius convention

The Ornstein–Zernike equations are *always* solved in reduced units (mean diameter unity): the radial grid spans only about 41 diameters, so a physical $\sigma = 100$ would place every hard core off the end of it. A mean radius supplied by the user is applied *after* the solve, to the diameters, the number densities ($n \propto L^{-3}$, which leaves φ invariant) and the q axis ($q_{\text{red}} = QL$); the tail parameters are already reduced and are untouched.

The scattering radius equals the hard-core radius, $R_i = \sigma_i/2$, for every class. Verified at $R = 50$: the $S(Q)$ peak moves to $Q = 0.06608$ while $Q\sigma = 6.608$ (hard spheres expect $\approx 2\pi$) and $QR = 4.496$ at the first $I(Q)$ minimum (exact sphere zero 4.493) — both unchanged from the dimensionless case, confirming a pure change of units.

Remark 7 (Known limitation). *Scattering size and interaction size are perfectly correlated by construction: no independent width, no offset. That is correct for charged colloids or bare silica, where the particle is the hard core; it is wrong for sterically stabilised particles whose brush is contrast-matched ($R < \sigma/2$), for charged colloids at low salt where the effective core exceeds the physical particle, and for solvated or partially matched shells.*

20 Closure suite extended and cross-validated

The closure suite was checked against `OrnsteinZernike.jl`, an independently written Julia library published with a peer-reviewed review. Bridge functions were compared at prescribed γ by evaluating $B = \ln g + \beta u - \gamma$ from this library’s $c(r)$.

Table 16: Bridge functions against the independent Julia implementation, measured at $\gamma > 0$. See the remark below: for BPGG and Duh–Haymet this range concealed a missing $\gamma < 0$ branch.

closure	$\max B_{\text{ours}} - B_{\text{Julia}} $
Verlet	1.1×10^{-16}
BPGG	1.1×10^{-16}
Bomont–Bretonnet	3.3×10^{-16}
Martynov–Sarkisov	2.2×10^{-16}
Charpentier–Jackse (CJVM)	identical formula
Zerah–Hansen (HMSA)	identical after substitution

Choudhury–Ghosh differs by parameterisation rather than error: this library uses a density-dependent coefficient $1.0175 - 0.275 \cdot 6\varphi/\pi$ whose zero-density limit is exactly Julia’s constant

$\alpha = 1.01752$.

Remark 8. *The agreement above was measured at $\gamma > 0$ only, and for BPGG and Duh–Haymet that hid a defect. Table I of the accompanying paper gives $b = -\gamma^2/2$ for $\gamma < 0$ for BPGG, MV, DH and CG; of these only CG implemented it here. BPGG instead continued its power law to negative γ through $\text{sign}(\cdot) \cdot |\cdot|^{1/s}$, which produces neither an overflow nor a NaN — so nothing appeared wrong — but is a numerical continuation rather than the published closure. Duh–Haymet was worse: its rational form is singular at negative γ^* , with poles at $\gamma^* = -9/7$ and $\gamma^* \approx -0.6$ and $\max |B| = \infty$.*

Both now take the published branch. The effect on results is small at low density and grows with coupling: for hard spheres, BPGG changes by 1×10^{-5} at $\varphi = 0.1$ and Duh–Haymet by 2×10^{-3} at $\varphi = 0.4$. A Yukawa fluid at $\varphi = 0.2$ is unchanged to 7×10^{-15} , which is the expected signature of a branch that fires only where it should. For $\gamma^ \geq 0$ no pole is possible in the Duh–Haymet form, since $7\gamma^* + 9 > 0$ and $(5\gamma^* + 11)\gamma^* \geq 0$, so the branch removes the singularities as well as matching the publication.*

The general point is that agreement to 10^{-16} over a restricted range of an argument says nothing about the rest of it. Cross-validation is only as good as the states at which it is performed.

Remark 9. *A first run reported Verlet as differing. The cause was mine: Julia’s default is $B = 8/5$ and it writes the denominator as $1 + B\gamma/2$, which is the same 0.8γ used here. Checking the actual defaults before reporting a discrepancy is the lesson; the same pattern — an apparent disagreement that is really an assumption of one’s own — recurs throughout this project.*

Three closures were added from that comparison: Khanpour ($B = \ln(1 + \alpha\gamma)/\alpha - \gamma$), Modified Verlet, and Extended Rogers–Young, whose bridge function is the ordinary RY construction with one extra quadratic term,

$$\phi = \frac{e^{f\gamma} - 1}{f}, \quad B = -\gamma + \ln(1 + \phi + a\phi^2), \quad (75)$$

so that $a = 0$ reduces *exactly* to RY — verified to 2.5×10^{-16} , an internal check independent of the Julia transcription. All three inherit the multicomponent path, every solver and the GUI dropdown automatically, because each is an elementwise expression in γ .

Remark 10 (Naming). *This library’s KH is Kovalenko–Hirata, which abbreviates identically to Khanpour but is a different closure; the new one is therefore spelled out in full. CJVM was confirmed to be Charpentier–Jackse, its formula matching Julia’s exactly.*

Availability by potential. Closures needing a one-component reference solve (Reference HNC, Modified HNC) or a hard-sphere-specific construction (Rescaled MSA, EuRah, ZSEP) cannot be used with a polydisperse potential, and the GUI now offers 19 of 23 rather than offering all and failing. Martynov–Sarkisov, Vompe–Martynov and CJVM are deliberately *not* hidden: they are structurally fine and fail only because their square-root bridge overflows $\exp(\gamma + B)$ for strongly coupled charged systems, a limitation that applies equally in one component.

21 Consistency search for a general potential

The scalar condition (70)–(72) is available for any potential built by the generic route. Two points differ from §15.2. First, $(\beta U)'$ is differentiated *numerically*, because the builder reuses arbitrary setters and cannot know their analytic derivative. Second, the reduced-tail potential (74) is density-independent, so perturbing φ leaves it untouched — which is not automatic, and is exactly the trap that produced a residual stuck near +65 in the charge-parameterised case.

The Rogers–Young residual crosses zero between $\alpha = 0.1$ and 0.5 and converges onto the HNC limit at large α (−4.1495 against HNC −4.1497) — exactly the behaviour that makes the closure

Table 17: Both routes against analytic Percus–Yevick hard spheres, $\eta = 0.3$. Residual differences are grid discretisation.

quantity	analytic	measured
χ^{-1}	10.6622	11.0678
$\beta P/\rho$	3.8163	3.8896
PY residual	+1.2482	+1.3284

work. A live search returns $\alpha = 0.2441$ with residual -6.9×10^{-5} .

Remark 11 (Judge the residual relatively). χ^{-1} is of order 10 for a dense fluid, so an absolute residual of 10^{-3} is a relative 10^{-4} . An absolute threshold rejected a perfectly good root ($\alpha = 0.2443$, residual -1.2×10^{-3}) as “no consistent value found”. The criterion is now relative to χ^{-1} .

Remark 12 (A false alarm worth recording). The search was first exercised at $s_\sigma = 0.2$, $p = 3$, $\varphi = 0.3$, where the residual saturated near +2.11 with almost no α dependence, and this was read as a broken virial route. With three moment-matched classes the largest diameter reaches $\sigma \approx 2.48$, so at $\varphi = 0.3$ the biggest spheres are at or past their own close packing: the state was unreachable, not the code wrong. Diagnosing a numerical defect from a single pathological point, without first checking a case with a known answer, was the error.

22 Defects found during validation

Each of the following produced plausible-looking wrong answers rather than an obvious failure, which is why they are recorded.

1. **HMSA returned $S(q) = 1$ everywhere** for hard spheres and for every potential except Lennard-Jones, Yukawa and DLVO. The cause: `repulsivePartOfP2Ppotential` and `attractivePartOfP2Ppotential` were never populated for those potentials, so $e^{-0} = 1$ gave no hard-core exclusion and $c(r) = 0$ became an exact fixed point. Fixed by a general fallback in `setPotentialByName()`. Independently re-confirmed afterwards: HMSA and RY are now bit-identical for hard spheres, as the physics requires (no attractive tail).
2. **Screening used Z^2 instead of Z** . Equation (66) involves the first power. The wrong version gives $g_{\max} \approx 1.006$, i.e. an essentially ideal gas.
3. **The virial route differentiated a density-dependent potential** (§15.2).
4. **KINSol reported success after diverging** (§16).
5. **KINSetMAA must precede KINInit**. The Python binding calls them in the opposite order; doing so in C segfaults on SUNDIALS 7.1.1, because the Anderson depth is set without the corresponding arrays being allocated. That Python tolerates it suggests the setting may simply be ignored there — in which case that path runs plain fixed-point iteration rather than the Anderson acceleration it appears to request. This is worth checking on the Python side.
6. **DST-I factor of two**. For an odd extension, $\text{Im } V[k+1] = -y[k]$; the factor 2 is already in the DFT.
7. **Warm starting silently did nothing**, because `ryp_build_potential` cleared the stored guess — precisely the case in which it should be kept.
8. **A loop that exhausted its iteration budget returned success**, because the status variable was overwritten by each call to the fixpoint map.

9. **Two placeholder “engines” that computed nothing.** Both the adhesive-sphere and the Rogers–Young Python cores shipped as stubs that returned plausible numbers without solving anything.

`robertus_shs_core_py.py`’s `solve()` body was a bare `pass` under a docstring reading “Dummy solve method for compatibility” — the multicomponent PY λ_{ij} system was never formed. `S_matrix()` started from `np.ones((p,p))` and filled *every* element, diagonal and off-diagonal alike, from a *single*-component expression $1/(1+24\varphi A)$ times an ad-hoc “sticky correction”, clamped with `np.maximum(S,0.1)` to force positivity. The size classes were therefore never coupled.

The diagnostic that exposed it is worth remembering: in the dilute limit an Ashcroft–Langreth S must tend to the *identity*. The placeholder gave diagonal ≈ 1 and *off-diagonal* ≈ 1 , i.e. an all-ones matrix. I_{exact} then collapses from $\sum_i \rho_i F_i^2$ to $(\sum_i \sqrt{\rho_i} F_i)^2$, inflating it by $(\sum_i \sqrt{w_i})^2$ — a factor depending only on the *number of classes*. Measured $I_{\text{exact}}/I_{\text{monodisperse}}$ at $s_\sigma = 0.01$, where every scheme must agree, was 1.00, 1.97, 2.53, 5.69, 8.84, 18.2 for 1, 2, 4, 8, 12, 24 classes, and the dilute limit gave 8.83 instead of 1.

This also explains the reported symptom that “the exact model does not respond to the distribution width”: polydispersity entered only through the class weights in that spurious sum, never through the physics. The approximation schemes call the real **Polydispers-eSASBase** machinery and so did respond — hence exact and approximate disagreeing in both scale and width dependence. Note the first diagnosis attributed this to a missing Ashcroft–Langreth conversion; that was wrong, since the diagonal was already correct, and only the off-diagonal test identified the true cause.

Replaced by a faithful port of the C engine (`py_compute_ab`, `py_residual`, `build_Q_matrix`, `rshs_structure_matrix`). Afterwards: dilute off-diagonal 2×10^{-7} , $I_{\text{exact}}/I_{\text{dilute}} = 0.99999$, $I_{\text{exact}}/I_{\text{monodisperse}} = 1.000$ for *every* class count, λ residual 2×10^{-16} , and $S(q)$ varying smoothly with width (0.7276, 0.7481, 0.8050, 0.8760 at $s_\sigma = 0.01, 0.1, 0.2, 0.3$).

Separately, `ry_polydisperse_core_py.py` had no density or volume fraction at all in its constructor, performed no OZ iteration, and built S_{ij} from an ad-hoc closed-form blend. It was replaced by an adapter onto the real multicomponent solver (§14.3).

10. **Unconverged and spurious solutions returned as answers.** `picardIteration()` prints “Picard did not converge after N steps” and then calls `derivePhysicalQuantitiesFromFixpoint()` regardless, exposing no flag, so callers cannot tell. For an attractive Yukawa ($K = 1$, $\varphi = 0.1$, $z = 2$) it returned a plausible $S(Q) = 2.65$ after diverging to a residual of 4.8×10^{204} .

Adding a residual check is necessary but *not sufficient*: the RY equations have multiple fixed points, and different solvers converge cleanly onto different ones. At that same state point the hand-written Anderson gave $S(0.5) = 17.03$ and `scipy` Anderson gave 9.71, *both* with residual $\sim 10^{-12}$ — and with $\min S_{NN}(q) = -9.19$ and -17.21 . A structure factor is a variance and cannot be negative, so both were spurious roots. A physicality screen ($\min S_{NN} \geq 0$) is therefore applied in addition to the residual check. Biggs–Andrews makes the same point from the other side: it declared convergence after 21 steps and failed the residual check.

11. **The Kronecker delta must not be interpolated.** Smearing δ_{ij} across a finer size grid turns the incoherent $\sum_i |F_i|^2$ into the coherent $|\sum_i F_i|^2$, reinstating exactly the form-factor oscillations that polydispersity should remove (§19.3). It presented as a physics question — “at $s_\sigma = 0.3$ no oscillations should be visible” — and was correct as such.
12. **A single class stored as a $(1, 1, N)$ array.** `isMulticomponent()` is false either way, so

the scalar code path ran on three-dimensional input and every derived curve came back shaped $(1, 1, N)$. `getRDF().max()` still returned the right number, which is precisely why the bit-identical $p = 1$ check passed and hid it; anything expecting a *curve* then failed with “object too deep for desired array”.

13. **A convergence test against the wrong array.** `x_0` holds the *start* value, not the converged fixpoint, so every monodisperse reference solve was rejected. It surfaced as all six approximation schemes reporting “no monodisperse MSA solution” — a message from the base class that looked like a physical obstruction rather than a caller’s bug.
14. **Two silent rejections in `setPotentialByName()`.** It counted arguments *with defaults* as mandatory, making any setter that carries optional parameters unreachable; and array-valued parameters appeared in the GUI as numeric boxes defaulting to 1.0, failing a length check. Both returned quietly after printing, leaving the potential untouched so that every closure produced the trivial $S(q) = 1$. The array parameters are now keyword-only, so `getfullargspec()[0]` — which the GUI builds its fields from — lists only the numbers a user can type.
15. **GUI defaults contradicting the documented physics.** Every potential parameter box was seeded with 1.0, so a user who left `chargeExponent` alone silently got $n = 1$ (charge linear in size) while the documentation promised $n = 2$ (constant surface charge density) — a different model, with nothing on screen to say so. Fields are now seeded from each setter’s own defaults.
16. **Tk touched from a worker thread.** Reading the solver dropdown inside the compute worker raised “main thread is not in main loop” and lost the entire run. Such controls are now read on the main thread and carried into the worker.
17. **Two convention errors, each worth about an order of magnitude.** (a) The Ashcroft–Langreth conversion (62) was omitted in the adhesive-sphere SAS layer, inflating I_{exact} by a factor that scales with the number of classes: the ratio $I_{\text{exact}}/I_{\text{monodisperse}}$ at $s_\sigma = 0.01$, where every scheme must agree, was 1.00, 1.97, 2.53, 5.69, 8.84 and 18.2 for 1, 2, 4, 8, 12, 24 classes, and the dilute limit gave 8.83 instead of 1. Because the number of classes is held fixed while sweeping the width, the exact curve sat about nine times too high and its genuine width dependence was visually swamped — the reported symptom was “the exact model looks independent of the distribution width”. (b) In the RY wrapper, $\exp(-z\sigma_i/2)$ was applied to `delta`, double-counting it because `PolydisperseOneYukawaMSA` takes δ_i directly. RY’s K -response came out a factor 7.2 too weak. Notably the *first* implementation was correct and a subsequent “correction” made from reading the specification introduced the error: only the numerical cross-check of Table 12 settled it.

23 Regression testing

The one-component regression suite covers eight combinations: PY, HNC, RY, HMSA and MSA on hard spheres; PY and RY on Yukawa; HNC on Lennard-Jones. Each records g_{max} , $\sum g$, $c(0)$, $\sum c$, S_{max} and $S(0)$. After the multicomponent extension all eight are **bit-identical** to the pre-change baseline. The suite is re-run against the deployed files rather than a working copy, and should be re-run after any change to the fixpoint operator.

24 Solver reliability: multiple solutions and verification

The work described above assumed throughout that a converged solve returns the right answer. That assumption does not hold in general, and the failure is not a defect in any particular solver.

24.1 The Ornstein–Zernike equation has several solutions

The number of solutions of the OZ equation with a given closure changes with density and temperature. Beardmore, Peplow & Bresme (SIAM J. Sci. Comput. **29**, 2442, 2007) analysed the bifurcation structure for a Lennard-Jones fluid under HNC: a subcritical isotherm carries at least one *fold bifurcation* in density, and pairs of branches collide at what they call pseudospinodals — one branch with positive compressibility, the other negative and therefore unphysical. They state that this explains “previous inconclusive attempts to compute solutions using Newton–Picard methods that are popular in the physical chemistry literature”, which is precisely what this package uses. Peplow *et al.* (Phys. Rev. E **74**, 046705, 2006) show further that solutions exist inside the so-called no-solution region which direct iteration cannot reach at all, and recover them by pseudo-arclength continuation.

We reproduce this. At $k_B T/\varepsilon = 1.25$, $\rho\sigma^3 = 0.57$ — well inside their problem region, and at lower temperature and roughly twice the density of their published examples — two of the five available solvers return $g_{\max} = 1.13$ and 1.43 against the correct 2.16 , finite, converged, and with no error raised. The 1.43 root has $\min S(Q) = -46.5$: it is not a failed calculation but a converged solution on the negative-compressibility branch the bifurcation analysis identifies.

24.2 What the library does about it

The physicality test is the discriminator, not a diagnostic. Since a structure factor is a variance it cannot be negative, and $\min S(Q) \geq 0$ is the same criterion the bifurcation studies use to reject branches. It is applied unconditionally. A user who disabled it would obtain a converged, finite, entirely wrong answer.

Verification by an independent solver, on by default. `ozLib.solve()` now takes `verify=True`: the solve is repeated with an independent solver from a *cold start* and the result returned only if the two agree on $S(Q)$ to 10^{-3} . Disagreement raises, naming both solvers and the size of the difference. `ozLib.solveWithConsensus()` is the more general form, running any number of solvers and requiring at least two to agree. Cost is two solves — 0.14 s against 0.05 s for hard spheres.

The verifier is chosen for *independence* rather than speed. Plain Picard iteration carries no acceleration and no history of previous iterates, so it cannot share an acceleration failure mode with the primary solver; Biggs–Andrews, which does keep a history, is the fallback when Picard fails.

The cold start cannot be optimised away. Warm-starting the verifier from the first answer is about three times faster and destroys the check. A wrong root is still a root: started from the erroneous 1.43 above, the otherwise reliable solver stays at 1.43 , the two “agree”, and the wrong answer is certified. The question the check must ask is whether independent searches from different starting points reach the same basin.

Limitation. Detecting disagreement is not the same as resolving it. Where a branch must be *followed* through a fold, pseudo-arclength continuation is the appropriate tool and is not implemented here.

24.3 Solver behaviour, measured

Measured over five test cases (hard spheres at $\varphi = 0.3$ and 0.45 , Lennard-Jones, square well, Yukawa):

Solver	solves	note
Anderson acceleration	5/5	correct, fastest (0.12s total)
Biggs–Andrews	5/5	correct
Picard iteration	5/5	correct
<code>scipy</code> Anderson	5/5	wrong root for Lennard-Jones
<code>scipy</code> Newton–Krylov	4/5	fails Lennard-Jones; slowest

Two corrections to earlier statements in this report follow from these measurements. First, Anderson acceleration does *not* require Picard pre-conditioning: it converges from $\Gamma = 0$ at every state point tested, and no pre-conditioning step exists anywhere in the code. Second, the Picard fallback was described as “damped” when it was not; it is now implemented in the averaged (Mann, or Krasnoselskii–Mann) form $\Gamma_{n+1} = (1 - \alpha)\Gamma_n + \alpha T(\Gamma_n)$ with $\alpha = 1$ as the default, so existing results are unchanged, and `mannAlpha` available to damp. The averaging matters: the Lennard-Jones case above diverges at $\alpha = 1$ and converges at $\alpha = 0.5$ on one discretisation, while on another the two exchange roles.

A third correction concerns cost rather than correctness. `scipy`’s Anderson implementation was the slowest solver available — 30s where others took 0.5s — purely because its initial inverse-Jacobian scale was left unset. Setting it makes that solver the fastest, a factor of 15 to 66, with the fixed point unchanged to 10^{-9} . Note that this is purely a cost effect: with the scale set, that solver converges to the same physical root as the other fixed-point methods on every case tested, so speed and branch selection are independent questions.

24.4 A converged flag is a claim; the residual is evidence

Throughout this section $\Gamma(r) = h(r) - c(r)$ denotes the indirect correlation function — the quantity every solver here actually iterates on, rather than $g(r)$ or $S(Q)$, because it is smooth across the hard core where h and c are not. $h(r) = g(r) - 1$ is the total correlation function and $c(r)$ the direct correlation function.

Writing the closure as $c = \mathcal{C}[\Gamma]$ and the Ornstein–Zernike relation in Fourier space as $\hat{\Gamma} = \rho \hat{c}^2 / (1 - \rho \hat{c})$, the two compose into a single *fixed-point map*

$$T(\Gamma) = \mathcal{F}^{-1} \left[\frac{\rho \hat{c}^2}{1 - \rho \hat{c}} \right], \quad \hat{c} = \mathcal{F}[\mathcal{C}[\Gamma]], \quad (76)$$

where $\mathcal{F}$ is the three-dimensional radial Fourier transform and ρ the total number density. A solution of the Ornstein–Zernike system is exactly a fixed point of this map, $T(\Gamma) = \Gamma$, and the *residual*

$$\|T(\Gamma) - \Gamma\|_2 \quad (77)$$

is therefore zero at a solution and nowhere else. For a multicomponent system the same holds with Γ , c and h replaced by $p \times p$ matrices Γ_{ij} , c_{ij} , h_{ij} , the scalar division by a matrix inverse, and ρ by $\sqrt{\rho_i \rho_j}$.

The distinction that matters below is that a solver’s own convergence flag reports whether *its own* stopping criterion was met — which may test the change between iterates, or their norms, or an internal estimate — while (77) tests whether the answer solves the equations. The two are not the same, and the difference is not academic.

The two situations above look identical from the outside — solvers disagreeing, every one of them reporting success — and are distinguished only by evaluating (77) at each claimed solution. Measured across all nine available solvers on a 4095-point grid:

State	Outcome	Residual
Lennard-Jones, $\varepsilon = 0.8$, $\varphi = 0.3$, HNC	5 fixed-point solvers, $g_{\max} = 2.1636$ 4 Newton–Krylov, $g_{\max} = 1.13$ – 1.22	$< 10^{-11}$ $< 10^{-11}$
BPGG $\alpha = 0.5$, $\varphi = 0.4$	8 solvers, $g_{\max} = 5.2528$ Picard, $g_{\max} = 3.0046$	$< 10^{-12}$ 19.4

The Lennard-Jones state has genuinely distinct solutions: all nine points are fixed points, and the Newton–Krylov family simply lands on branches with $\min S(Q) \approx -38$, which are unphysical but real. The BPGG state has ONE solution; the ninth value is not a fixed point at all.

The cause of that last case was a defect in the Picard convergence test, which compared successive *norms* of Γ rather than the change in Γ . Two different vectors of similar norm pass such a test, so an iterate still moving but whose norm has momentarily plateaued is reported as converged. It now tests the increment, and exposes a **converged** attribute that defaults to false, since exhausting the iteration limit previously returned the last iterate with nothing a caller could check.

Two practical consequences. First, recompute the residual after every solve; it costs one operator evaluation and is the only thing that distinguishes the two rows above. Second, when using one solver to verify another, note that **Picard iteration** and **Biggs-Andrews** converge to residuals of $\sim 5 \times 10^{-11}$ on hard spheres where the others reach 10^{-13} – 10^{-14} ; a verifier’s own tolerance sets the floor on the disagreement it can detect.

Where the question is not whether a point is a solution but whether OTHER solutions exist, the appropriate tool is deflation [?]. Given a known root r of the residual $F(\Gamma) = T(\Gamma) - \Gamma$, one forms the *deflated residual*

$$G(\Gamma) = M(\Gamma; r) F(\Gamma), \quad M(\Gamma; r) = \frac{1}{\|\Gamma - r\|^p} + \alpha, \quad (78)$$

and re-solves *from the same initial guess*. The exponent p (here $p = 2$) controls how sharply the known root is excluded, and the shift α (here $\alpha = 1$) is essential rather than cosmetic: with $\alpha = 0$ the deflated residual tends to zero as $\|\Gamma - r\| \rightarrow \infty$, so a solver can report convergence having simply run away. Because M is finite and non-zero at every other root, those are left reachable, and repeating with one factor per known root discovers them in turn.

The method is not a guarantee — it finds distinct solutions where they exist and the solver converges, but does not certify that none remain — and the authors note it is sensitive to α , with no a priori guidance. Applied to the BPGG state above, deflating the single root yields nothing further, consistent with there being only one.

25 Fitting measured data

What is fitted is

$$I(Q) = \mathcal{R} \left[\text{scale} \times I_{\text{model}}(Q) + A Q^{-4+d} \right] + \text{background}, \quad (79)$$

where $\mathcal{R}$ is the resolution smearing. The power-law term describes the sloping background that real curves frequently carry — Porod’s Q^{-4} for a smooth sharp interface, shallower for a rough or fractal one. With only a flat constant available the fit absorbs that slope into the shape parameters, where it does real damage.

Note which terms are inside $\mathcal{R}$. The power law is scattering and is smeared; the incoherent background is flat and unaffected by smearing, so it is added afterwards.

Linear terms are solved, not searched. The scale, the constant and the power-law amplitude all enter (79) linearly, so at every iteration they are obtained exactly by weighted linear least squares over whichever of them are free, while the nonlinear optimiser sees only the shape parameters and the exponent d . Giving them to the optimiser instead would add the three most strongly correlated quantities in the problem — scale against volume fraction, background against everything at high Q , the power law against both — to a search that gains nothing by including them. Enabling the power law therefore costs ONE extra fitted parameter, not two. Any of the three may also be held fixed, in which case the remainder are still solved exactly; a held term simply moves to the right-hand side.

The resolution kernel is Rician, not Gaussian. A small-angle instrument broadens Q in the detector plane, in two dimensions, and the data are then radially averaged. For isotropic Gaussian broadening of width σ the distribution of the *magnitude* about a true Q_0 is

$$P(Q|Q_0) = \frac{Q}{\sigma^2} \exp\left(-\frac{Q^2 + Q_0^2}{2\sigma^2}\right) I_0\left(\frac{QQ_0}{\sigma^2}\right), \quad (80)$$

with I_0 the modified Bessel function of the first kind. A Gaussian in Q is the $Q_0/\sigma \gg 1$ limit of this and is adequate wherever dQ/Q is small. It is not always small: for the spherical-shell data used to develop this, dQ/Q reaches 0.55 at the lowest measured point, where a Gaussian places 44 % of that point's kernel weight below $Q_{\min}$ and some of it at *negative* Q . The leading factor of Q in (80) suppresses the origin, reducing that to 34 % and excluding negative Q by construction. Both forms reproduce the unsmeared curve to 2×10^{-16} as $dQ \rightarrow 0$, which is the test that catches a mis-normalised kernel. (80) is evaluated through the exponentially scaled $\tilde{I}_0(x) = e^{-|x|} I_0(x)$, since the argument reaches several hundred at these widths and I_0 overflows a double long before the prefactor cancels it.

The power law must not be extrapolated to $Q \rightarrow 0$. Q^{-4+d} diverges at the origin while the smearing kernel reaches below the data, so something has to bound it. Left unbounded the result is set by whatever guard prevents division by zero rather than by the physics: with a Gaussian kernel the smeared power law at the lowest measured point came out 6.6×10^5 times the unsmeared value, and moving that guard from $Q_{\min}/1000$ to $Q_{\min}/10$ changed the answer by five orders of magnitude. The term is therefore held constant below a Q the user sets. Choosing the lowest measured Q suppresses the extrapolation entirely; the default is far below it, which keeps the power law extending into the unmeasured region. That is a physical judgement — whether the scattering really continues to rise below the window — and is exposed rather than decided in the code. It is worth fitting both ways: if A and d are stable the choice does not matter, and if they swing then the lowest points are driving the result.

Linking the interaction range to the shell. For the square-well and sticky-sphere potentials the second argument is the well WIDTH as an absolute length, directly comparable with a core-shell particle's shell thickness. Where the attraction comes from the surface layer the two are the same physical quantity, and fitting them independently invites them to trade against one another. They can therefore be linked as $\delta = c \Delta R$, with δ recomputed from the current ΔR at every iteration and removed from the fitted set. The constant c may itself be fitted, bounded between 0 and 2: $c = 1$ if the range is the layer, $c = 2$ if two layers overlap when particles touch.

The contrast degeneracy is removed, not refused. The intensity depends on the scale times the squared contrast, so the absolute scattering length densities carry no information the scale does not. The fitted contrast parameter is therefore the `RATIO` $\rho_{\text{shell}}/\rho_{\text{core}}$, which the interface exposes as its own field alongside the two densities; editing any of the three updates the others. Fitting the ratio also makes the parameter refine at all: in cm^{-2} a scattering length density is $\sim 10^{10}$, and `least_squares` satisfies its gradient test after a single evaluation at that magnitude, returning the starting value with a meaningless uncertainty. A ratio of order one has no such problem. Its bounds span both signs and are wide, so that an opposite sign between core and shell is an outcome of the fit rather than an imposition.

Ill-conditioning in the linear solve. The three linear coefficients are obtained from a weighted least-squares problem whose design columns can differ in magnitude by twenty orders: a model built from scattering length densities in cm^{-2} carries contrast squared and so has magnitude $\sim 10^{20}$, while the constant background column is unity. `numpy.linalg.lstsq` discards singular values below $\max(M, N)\epsilon$, so at a condition number of 10^{20} the background and power-law coefficients are truncated to EXACTLY zero and the solve reports success. The symptom is a fit returning `background = 0` and `A = 0` that a hand-chosen background of 0.5 and amplitude of 2×10^{-3} beats by 30 % in χ^2 — which looks like a broken optimiser and is a discarded singular value. Equilibrating the columns to unit norm before the solve and rescaling afterwards restores all three coefficients to five figures, and leaves well-conditioned problems unchanged.

Bounds are shown, editable, and not silently widened. Every fittable parameter carries its own *vary* checkbox and a pair of bound fields beside its value, greyed together when the parameter does not apply to the current model. Showing the bounds matters even unedited: a parameter pinned at a limit is a result — the data want to go where the model forbids — and that is invisible if the limit is. Editing them is how independent knowledge enters: a radius from electron microscopy or a volume fraction from weighing constrains the fit far better than a generic range.

The built-in limits are treated as the maximum `DOMAIN` rather than as defaults to be overridden, so an edited bound may narrow the range and not widen it, and a value outside its own bounds is refused rather than clipped. Both are reported as errors. Silently moving a starting value to a boundary, or honouring a limit the user typed only when it happens to be convenient, makes a constraint into a suggestion — and a limit that looks respected and is not is worse than no limit at all.

Optimisers. The default is `scipy.optimize.least_squares`, a bounded trust-region method that converges in some thirty model evaluations. Since one evaluation is a full Ornstein–Zernike solve — 0.05 s for one component, 2.5 s at seven size classes — the population methods available through `bumps` (differential evolution, DREAM) are hours rather than minutes and are not everyday alternatives. They earn their place in two situations: DREAM samples a posterior and returns a correlation matrix, which is the only honest way to see how many of eleven nominally free parameters the data actually determine; and a population method can cross between minima that a local one cannot, which matters when a sign or a branch is in question. A multi-start option answers the same question for about three hundred evaluations rather than several thousand.

`NLopt` is offered for one reason only: constraints that `RELATE` several parameters, which no box bound can express. The concrete one here is that the shell must fit inside the core — and not inside the mean core but the smallest, since the distribution has width *s* and the smallest

class has radius about $R(1 - ns)$:

$$\Delta R < R(1 - ns). \quad (81)$$

Without it an unconstrained fit on synthetic data drove the shell to 0.84 against a limit of 0.63 — thicker than the core it sits on — while the constrained fit found the right answer with a LOWER χ^2 . AUGLAG wraps a derivative-free inner method, so the constraint costs no extra evaluations per step.

Warm starting. Each solve may be seeded with the previous converged Γ rather than started from zero. Measured at 1.2 times faster with 20% fewer solver iterations and an identical fit result, and the gain grows with the cost of a single solve. It also keeps the fit on one branch of the closure equations, which makes the residual smooth in the parameters where a cold start can land on a different branch between adjacent iterations. It is off by default, because it can equally carry a fit ONTO a wrong branch and hold it there; the residual check and the $\min S(Q) \geq 0$ screen bound that risk but do not remove it.

What a session file holds. Settings, the measured data including its resolution column, which parameters were free, any edited bounds, and the last fit result — everything needed to resume. It does NOT hold the computed runs: those are regenerated by pressing Compute, and storing them was expensive out of all proportion. One partial structure factor is shaped (n_Q, p, p) , which at 200 q -points and 40 classes is 320 000 values per run; thirty runs of that produced a 196 MB file, large enough to fill a disk after a few saves. Written without the runs and indented for legibility the same session is 23 kB.

Edited bounds travel with the file deliberately. A limit narrowed from an independent measurement is knowledge about the sample rather than a preference, and losing it on reload would quietly return the fit to the generic defaults.

Reproducibility of the external comparisons. Several rows of the validation table were measured against `jscatter`, whose dependency chain — MDAnalysis, JupyterLab, ngview, pdb2pqr, netCDF4 — does not build under MinGW. Those comparisons were therefore not repeatable on the machine this work is written on, which is an uncomfortable position for a validation claim: a reader cannot check it, and neither could we.

The two-Yukawa solver has been extracted into `tools/two_yukawa_reference.py`, needing `numpy` alone, with `tools/two_yukawa_test.py` performing the comparison and writing the result as JSON. That row is now reproducible locally, and the exercise sharpened it: the quoted 0.15–0.28% became a single 0.21% at 1600 points per diameter, with successive convergence ratios of 4.22 and 4.24 confirming first order twice over. The earlier spread was resolution, not uncertainty.

Provenance is recorded in the module header rather than left implicit: the derivation is ?], the algorithm their own MATLAB by way of C, Igor and `jscatter`, and the licence descends from the NIST SANS reduction package, described there only as freely available and formally unclear. The paper asks to be cited by anyone using results from the code, and that request is passed on. It follows that this is NOT an independent check of the published equations — only of our discretisation against the authors' own implementation, which is a weaker and more honest claim.

The remaining `jscatter` rows are in the same position and deserve the same treatment before submission.

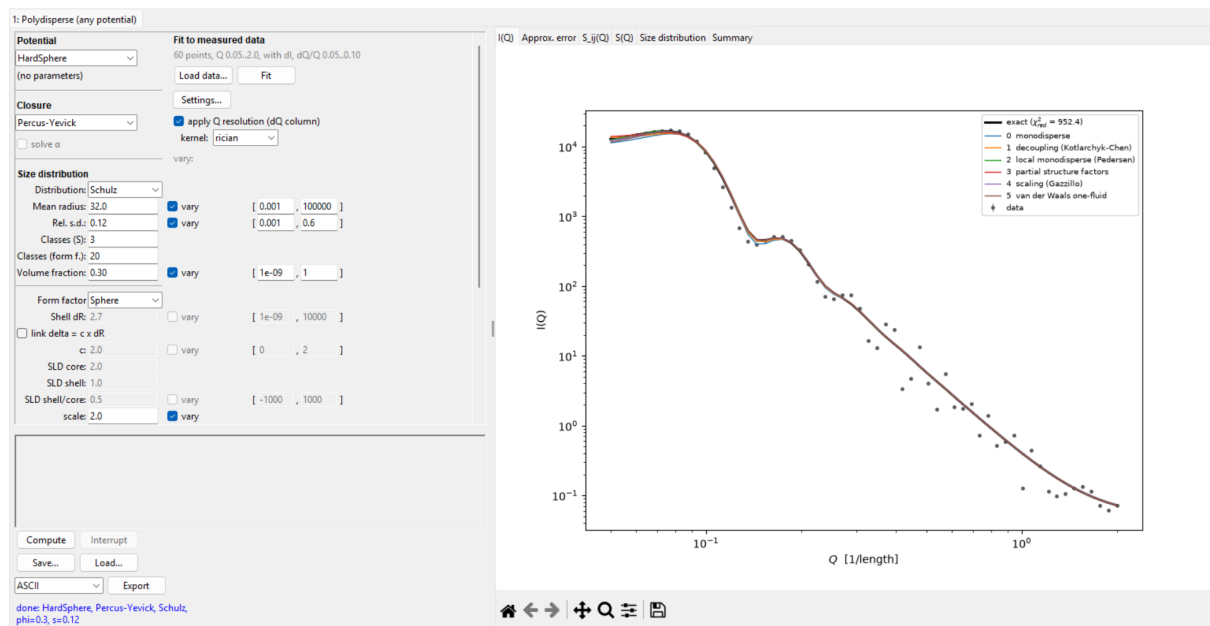


Figure 8: The parameter panel. Each fittable quantity carries its value, a *vary* checkbox and its own pair of bounds on one row, so nothing has to be matched against a separate list. The bounds are editable: the built-in limits shown here are the maximum *domain*, which an independent measurement may narrow but not widen. Parameters that do not apply to the current model are greyed across all four fields — the interaction-range ratio c here, because the $\delta = c \Delta R$ link is switched off — rather than hidden, so the layout does not reflow as options are toggled. The divider between panel and plots is draggable.

Regression tests for the silent failures. `tools/numerics_test.py` guards four numerical defects that shared a shape: the wrong answer looked plausible, nothing raised, and the only way to see it was to compute a number and compare it with what it should have been. A test that merely ran the code would have passed in every case, so these assert VALUES — 27 checks, all passing.

The linear solve is checked at a condition number of 1.8×10^{20} , where it must recover the scale, background and power-law amplitude to five figures rather than returning two of the three as exact zeros, and the non-negativity constraint must bind at exactly zero rather than nearly. Both resolution kernels must reproduce the unsmeared curve as $dQ \rightarrow 0$ (measured 2.2×10^{-16}) and normalise to unity, and the Rician form must place less weight below $Q_{\min}$ than the Gaussian — 0.264 against 0.388, which is the whole reason for preferring it and is therefore asserted rather than assumed. All five size distributions must reproduce their mean and width to six figures. The power-law clamp must leave the smeared value unchanged when the arbitrary low- Q guard is moved by two orders of magnitude (0.42% spread, against five orders unclamped) and must be invisible away from that edge, since a test of stability alone could be satisfied by clamping everything flat.

A fifth block exercises the DREAM posterior, wired for some time and never once run to completion. It completes: 3697 evaluations, none failed, the uncertainties populated and the correlation matrix of the right shape with unit diagonal. The concern was specific — the state lookup sits inside a bare `except Exception: pass`, so a wrong attribute path would have removed the correlation matrix with no message at all — and it is unfounded. The posterior itself remains unvalidated: the test fits noiseless synthetic data, so the returned uncertainties are essentially the prior widths and the correlation is near zero where two parameters would be expected to couple. That needs one run on measured data.

25.1 Where the hard core sits on the grid

The most consequential numerical finding of this work is also the least visible one, because nothing about it looks wrong: every solve converges, the residual reaches 10^{-12} , and the resulting $S(Q)$ is smooth, positive and physically plausible. It is simply the wrong curve by a few per cent at low Q , and no amount of refining the grid in the usual way removes it.

The diagnostic that exposed it is the one quantity for which an exact answer exists: the Percus–Yevick compressibility for monodisperse hard spheres,

$$S(0) = \frac{(1 - \varphi)^4}{(1 + 2\varphi)^2}. \quad (82)$$

Extrapolating the computed partials to the origin and comparing gives an ABSOLUTE accuracy, where every other check in this work is a comparison between two approximations.

Table 18: Relative error in $S(0)$ against Eq. (82), monodisperse hard spheres. *pps* is radial points per diameter; the grid size is scaled with it so that the real-space range is unchanged.

φ	transform	pps = 100	pps = 400
0.30	type 1	3.67 %	0.92 %
0.30	type 4	0.015 %	0.003 %
0.40	type 1	5.27 %	1.33 %
0.40	type 4	0.036 %	0.004 %

The cause. A DST-I places its nodes at $r = (n + 1)\Delta r$, so the hard core at $r = \sigma$ falls exactly ON a node. The discontinuity is then carried by a point that is neither inside nor outside, and the error in the excluded volume is first order in Δr . A DST-IV places them at $(n + \frac{1}{2})\Delta r$: the core falls BETWEEN points and the step is resolved to second order.

Table 18 shows exactly that. Type 1 falls from 5.27 to 1.33 % for a fourfold refinement, a ratio of 3.96 — textbook first order. Type 4 starts at 0.036 % on the COARSE grid, already 150 times better than type 1 achieves at four times the cost, and reaches 0.004 %.

Why it took so long to find. The error is independent of the real-space range. Extending $r_{\max}$ from 41σ to 655σ — a sixteenfold increase — changed $S(0)$ in the sixth decimal and the error not at all. Refining the resolution at fixed range does reduce it, but only fourfold per step, so a convergence study that looks at ratios sees respectable first-order behaviour and concludes the method is working. It is: just slowly, towards an answer whose leading error is set by where a discontinuity happens to land.

It also explains a discrepancy that had stood open. The mixture validation tab reports a numerical solve that is a fixed point to 7×10^{-12} — machine precision, so not a solver tolerance — and yet differs from two independent analytic references by about 10^{-3} , with the difference barely shrinking under refinement and growing with φ . That is this same effect measured from the other side, and the two were one problem.

What it means for a fit. $S(0)$ is the isothermal compressibility, so an error there is not a cosmetic one at an unmeasured point: it is a systematic bias in the quantity the volume fraction controls. A fit will move φ to compensate, and the compensation is larger the denser the sample — precisely where an interacting model is being used because it matters.

The limitation, which is now much more serious than it looked. Type 4 requires every pair core to fall between grid points. For one size class that is arranged by construction. For a MIXTURE it cannot be: the pair diameters $\sigma_{ij} = (\sigma_i + \sigma_j)/2$ come from a moment-matched quadrature and are mutually incommensurate, so no single grid offset places them all between nodes. The polydisperse route is therefore restricted to type 1 and carries the first-order error above.

This was previously recorded as a modest inconvenience — type 4 being “more accurate but not available for mixtures”. Table 18 reframes it: the difference is not a refinement but the difference between 0.03 % and 5 % in the compressibility. Until a mixture-capable second-order treatment exists, a polydisperse fit at $\varphi \gtrsim 0.3$ should be run at the finest radial resolution affordable, and its fitted volume fraction read with this bias in mind.

26 Known limitations

1. Closures requiring an auxiliary reference solve (RHNC, MHNC, RMSA, EuRah) remain one-component.
2. Closures reading the repulsive/attractive potential split (HMSA, VM, CJVM, BB, DH, CG, SMSA) are not yet wired for the multicomponent potential; those arrays remain 1-D. PY, HNC, RY, Verlet and BPGG work. RY is the one that matters here.
3. `ryp_SM()` interpolates linearly between q -grid points, which underestimates the main peak by about 0.05%.
4. The plugin’s cache is a single static and therefore not thread-safe. SASfit evaluates a model serially in q , which matches how it is driven.
5. The contact-value extrapolation in (72) is the least certain step of the consistency search.
6. A thermodynamically consistent α does not exist everywhere in parameter space; this is reported rather than hidden.
7. Charges are effective (renormalised), not bare (§14.2).